

Companion to Bird & de Moor, Algebra of Programming

§9. Dynamic Programming

§9.1. Theory

§9.1.1. Example: segmenting a list

- (9.1.1a)
- $\langle T \rangle = \text{concat}$
 $T = [\text{nil}, \text{cat}]$ takes a non-empty first list, so T° cuts a non-empty prefix off a list every way, and folding with T flattens a list of segments
 - $\langle h \rangle = \text{list}(\text{sum})$
 $h = [\text{nil}, \text{cons}(\text{sum} \times 1)]$ puts a segment's sum in front, so folding with h sums every segment
 - $H = \text{partition list}(\text{sum})$
 H segments a list every way, then sums each segment; with R comparing the largest entry, M cuts the list so that the largest segment sum is as small as possible

(9.1.1b)

x	h(x)
1 l(())	[]
2 r([c],[])	[c]
3 r([a, b],[c])	[a + b, c]
4 r([a],[b + c])	[a, b + c]

- (9.1.1c)
- $H([a, b, c], ys) \iff ys = [a + b + c] \vee ys = [a, b + c] \vee ys = [a + b, c] \vee ys = [a, b, c]$
 H sends $[a, b, c]$ to the sums of its four segmentations and nothing else
 - $T^\circ F(H) h = H$
cut once, solve the rest, put the pieces back together

(9.1.1d)

name	type	definition	meaning
1 H	$A \rightarrow B$	$H \triangleq \langle T \rangle^\circ \langle h \rangle$	take the input apart by the T -decompositions, and put it together by h
2 M	$A \rightarrow B$	$M \triangleq \frac{H}{\exists} \text{est}(R)$	the problem to be solved: an answer of H that is best under R

Map(h) $\wedge$ F(R $^\circ$) $h \sqsubseteq h$ R $^\circ \wedge$ R $^\circ$ R $^\circ \sqsubseteq$ R $^\circ \implies (\mu X : \frac{T^\circ}{\exists} P(F(X) h) \text{est}(R)) \sqsubseteq \frac{H}{\exists} \text{est}(R)$

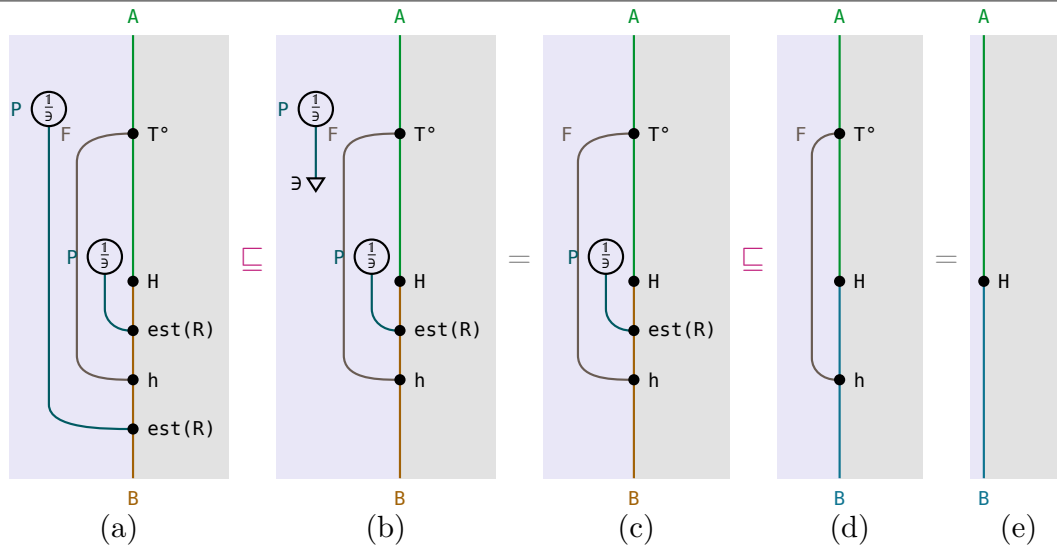
Theorem 9.1

every answer the recursion returns is an optimal one

- $\frac{T^\circ}{\exists}$ takes the input apart one step every way; $F(X)$ solves each part by the recursion X ; h assembles each candidate; $\text{est}(R)$ keeps a best one
- $\sqsubseteq M$: that never misses the optimum, though all candidates are never listed as H would
- h monotonic on R : a better part never makes the whole worse
- $\sqsubseteq$, not $=$: some optima may not be returned; that the recursion returns anything at all needs T° to stop taking apart — dynamic programming theorem

$$\frac{T^\circ}{\exists} P(F(\frac{H}{\exists} \text{est}(R)) h) \text{est}(R) \sqsubseteq H \quad (9.2)$$

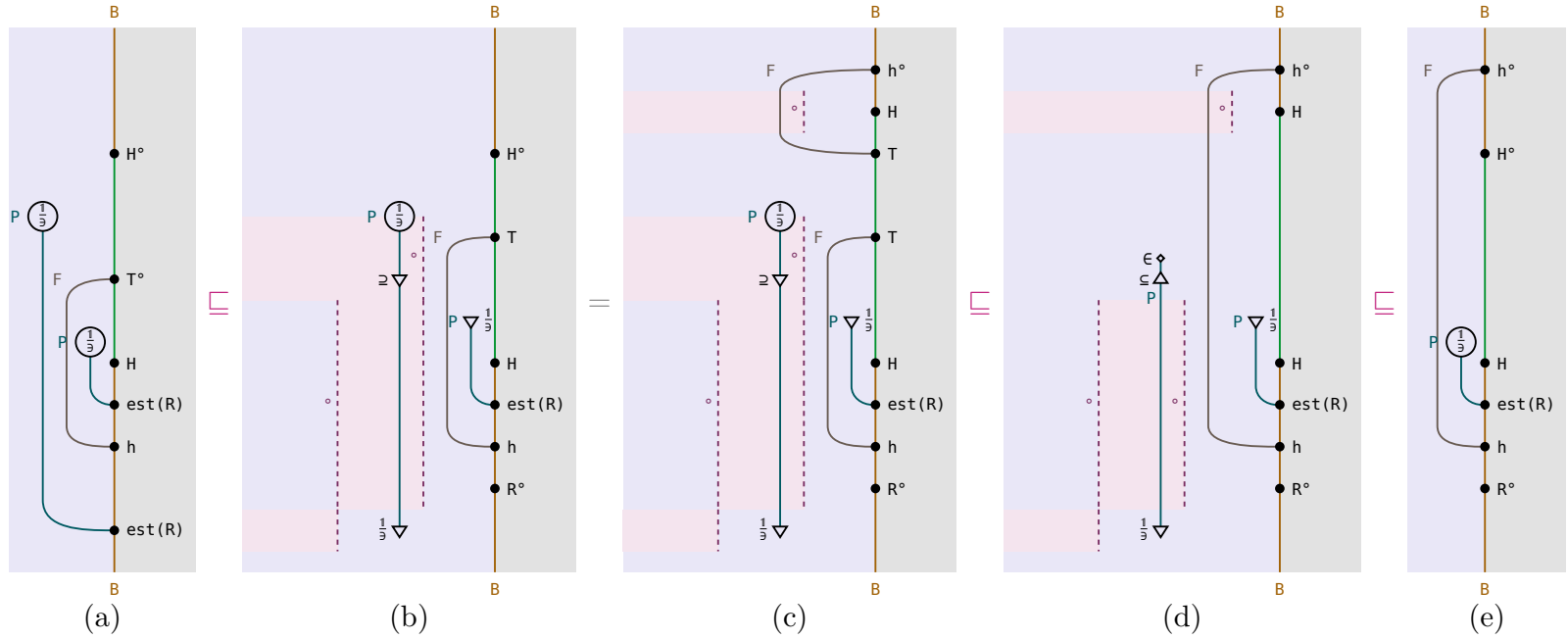
taking the input apart every way T allows ($\frac{T^\circ}{\exists}$), solving each part by M and keeping an optimum ($P(F(M)h) \text{est}(R)$)
returns only what H returns



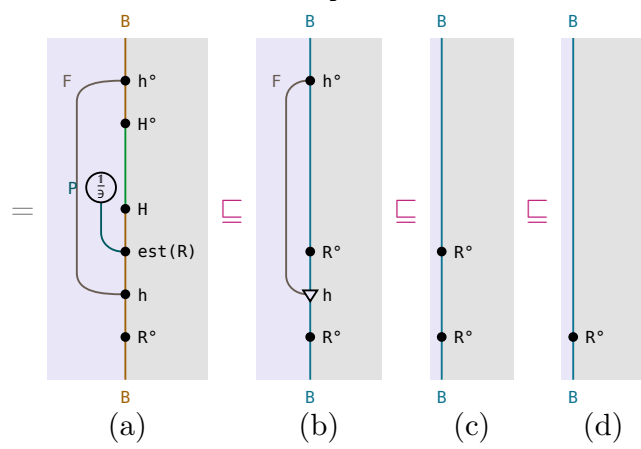
(a)	$\frac{T^\circ}{\exists} P(F(\frac{H}{\exists} \text{est}(R)) h) \text{est}(R)$	$P(S) \text{est}(R) \sqsubseteq \exists S$
(b)	$\sqsubseteq \frac{T^\circ}{\exists} \exists F(\frac{H}{\exists} \text{est}(R)) h$	$\frac{R}{\exists} \exists = R$
(c)	$= T^\circ F(\frac{H}{\exists} \text{est}(R)) h$	$\frac{S}{\exists} \text{est}(R) \sqsubseteq S$
(d)	$\sqsubseteq T^\circ F(H) h$	$T^\circ F(H) h = H$
(e)	$= H$	

$$\text{Map}(h) \wedge F(R^\circ) \ h \sqsubseteq h \ R^\circ \wedge R^\circ \ R^\circ \sqsubseteq R^\circ \implies H^\circ \xrightarrow{\frac{T^\circ}{3}} P(F(\frac{H}{3} \text{ est}(R)) \ h) \text{ est}(R) \sqsubseteq R^\circ \quad (9.3)$$

for every $\mathbf{b}$ that $\mathbf{H}$ returns from an input, the step $\frac{T^\circ}{3} P(F(\mathbf{M})h) \text{ est}(R)$ returns from that input only $\mathbf{b}'$ with $\mathbf{R}$ relating $\mathbf{b}'$ to $\mathbf{b}$



(a)	$H^\circ \xrightarrow{\frac{T^\circ}{3}} P(F(\frac{H}{3} \text{ est}(R)) \ h) \text{ est}(R)$	$P(S) \text{ est}(R) \sqsubseteq \epsilon \setminus (S \ R^\circ)$
(b)	$\sqsubseteq H^\circ \xrightarrow{\frac{T^\circ}{3}} (\epsilon \setminus (F(\frac{H}{3} \text{ est}(R)) \ h \ R^\circ))$	$H^\circ = h^\circ F(H^\circ) \ T$
(c)	$= h^\circ F(H^\circ) \ T \xrightarrow{\frac{T^\circ}{3}} (\epsilon \setminus (F(\frac{H}{3} \text{ est}(R)) \ h \ R^\circ))$	$T \xrightarrow{\frac{T^\circ}{3}} \epsilon$
(d)	$\sqsubseteq h^\circ F(H^\circ) \ \epsilon \ (\epsilon \setminus (F(\frac{H}{3} \text{ est}(R)) \ h \ R^\circ))$	$S \ (S \setminus R) \sqsubseteq R$
(e)	$\sqsubseteq h^\circ F(H^\circ) \ F(\frac{H}{3} \text{ est}(R)) \ h \ R^\circ$	



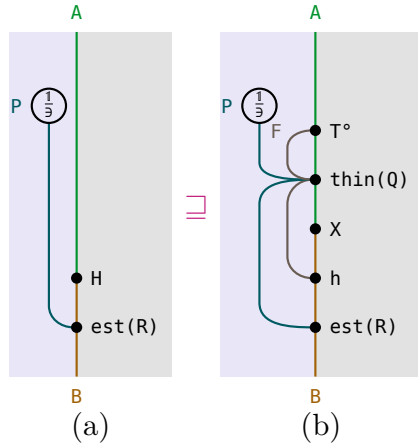
(a)	$= h^\circ F(H^\circ \xrightarrow{\frac{H}{3}} \text{ est}(R)) \ h \ R^\circ$	$S^\circ \xrightarrow{\frac{S}{3}} \text{ est}(R) \sqsubseteq R^\circ$
(b)	$\sqsubseteq h^\circ F(R^\circ) \ h \ R^\circ$	$\text{Map}(f)$
(c)	$\sqsubseteq R^\circ \ R^\circ$	$\implies F(R) \ f \sqsubseteq f \ R \iff f^\circ F(R) \ f \sqsubseteq R$
(d)	$\sqsubseteq R^\circ$	$R^\circ \ R^\circ \sqsubseteq R^\circ$

Map(h) $\wedge$ F(R^o) $h \sqsubseteq h R^o \wedge R^o R^o \sqsubseteq R^o \wedge \text{ThinCondition}(T, h, R, Q) \Rightarrow (\mu X : \frac{T^o}{3} \text{thin}(Q)$ Theorem 9.2

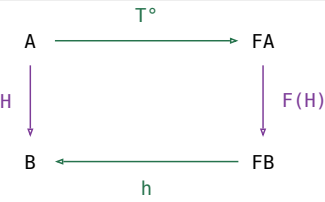
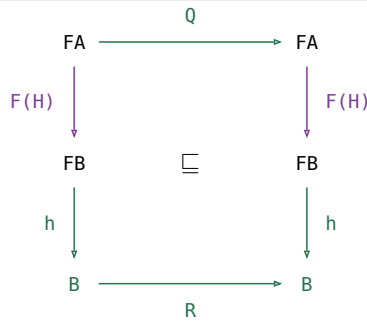
$$P(F(X) h) \text{est}(R)) \sqsubseteq \frac{H}{3} \text{est}(R)$$

an optimum over everything **H** returns is reached by taking the input apart every way **T** allows, dropping the parts that can never win, solving each of the rest and keeping one optimum

$$H \triangleq (T)^o \circ (h) \quad H \triangleq (T)^o \circ (h)$$



(a)	$\frac{H}{3} \text{est}(R)$	$QF(H)h \sqsubseteq F(H)hR, h \text{ monotonic on } R \text{ — } Q \sqsubseteq R \wedge 1 \sqsubseteq Q \wedge R R \sqsubseteq R$ $\Rightarrow \text{est}(R) = \text{thin}(Q) \text{est}(R), X \sqsubseteq \text{est}(R) \Leftrightarrow X \sqsubseteq \exists \wedge \in X \sqsubseteq R^o$
(b)	$\sqsubseteq \frac{T^o}{3} \text{thin}(Q) P(F(X) h) \text{est}(R)$	



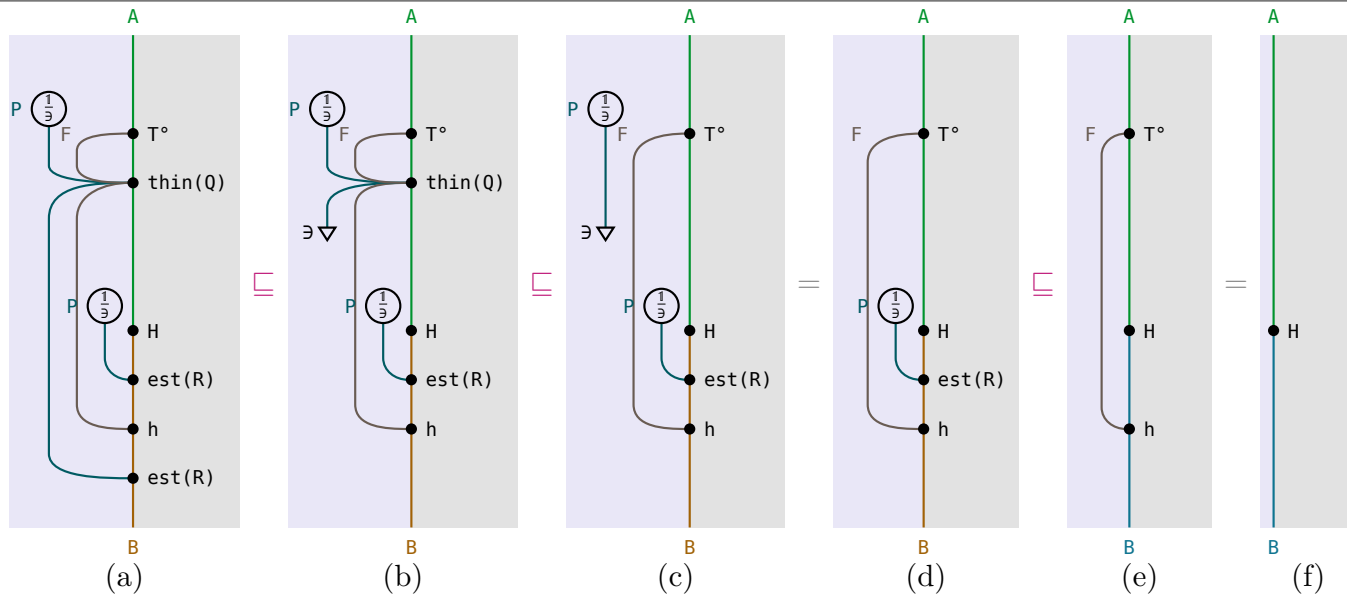
$T^o F(H)h = H$: taking the input apart once by T^o , solving the parts by $F(H)$ and assembling by h returns exactly what H returns

$QF(H)h \sqsubseteq F(H)hR$: every result of thinning by Q , solving the parts by $F(H)$ and assembling by h is reached by R from a result of solving by $F(H)$ and assembling by h without Q

$$\frac{T^o}{3} \text{thin}(Q) P(F(\frac{H}{3} \text{est}(R)) h) \text{est}(R) \sqsubseteq H$$

(9.1.1e)

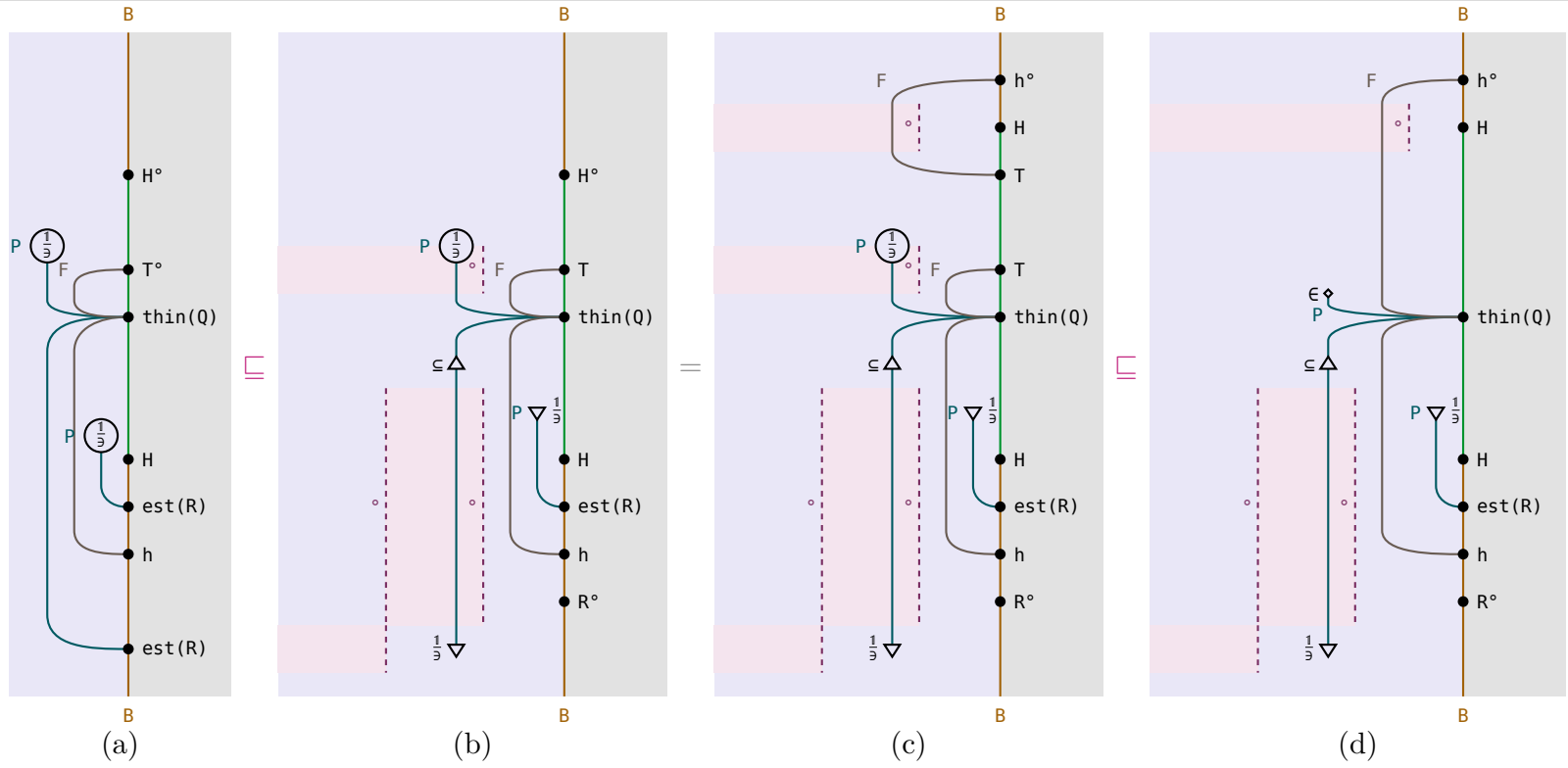
taking the input apart every way **T** allows ($\frac{T^o}{3}$), dropping the parts **thin(Q)** rejects, solving each rest by **M** and keeping an optimum returns only what **H** returns



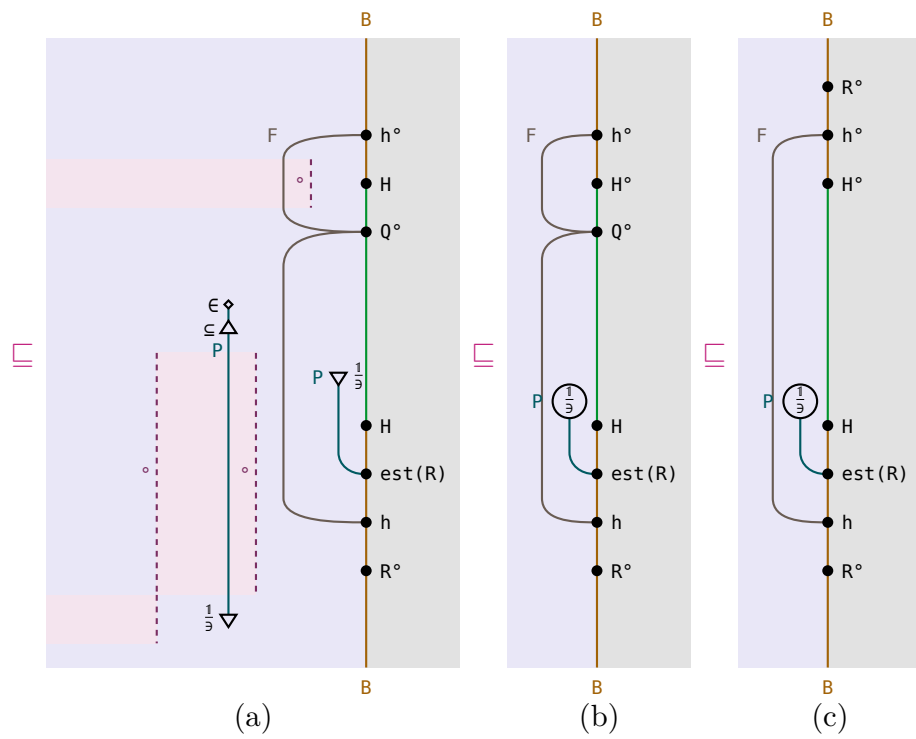
(a)	$\frac{T^o}{3} \text{thin}(Q) P(F(\frac{H}{3} \text{est}(R)) h) \text{est}(R)$	$P(S) \text{est}(R) \sqsubseteq \exists S$
(b)	$\sqsubseteq \frac{T^o}{3} \text{thin}(Q) \exists F(\frac{H}{3} \text{est}(R)) h$	$\text{thin}(Q) \exists \sqsubseteq \exists$
(c)	$\sqsubseteq \frac{T^o}{3} \exists F(\frac{H}{3} \text{est}(R)) h$	$\frac{R}{3} \exists = R$
(d)	$= T^o F(\frac{H}{3} \text{est}(R)) h$	$\frac{S}{3} \text{est}(R) \sqsubseteq S$
(e)	$\sqsubseteq T^o F(H) h$	$T^o F(H) h = H$
(f)	$= H$	

$$\text{Map}(h) \wedge F(R^\circ) \ h \sqsubseteq h \ R^\circ \wedge R^\circ \ R^\circ \sqsubseteq R^\circ \wedge \text{ThinCondition}(T, h, R, Q) \Rightarrow H^\circ \frac{T^\circ}{\exists} \text{thin}(Q) \ P(F(\frac{H}{\exists} \text{est}(R)) \ h) \ \text{est}(R) \sqsubseteq R^\circ \quad (9.1.1f)$$

H° followed by the body at $\mathbf{M}$ is $\mathbf{E}R^\circ$: an answer of the body is never worse than an answer of $\mathbf{H}$ to the same input

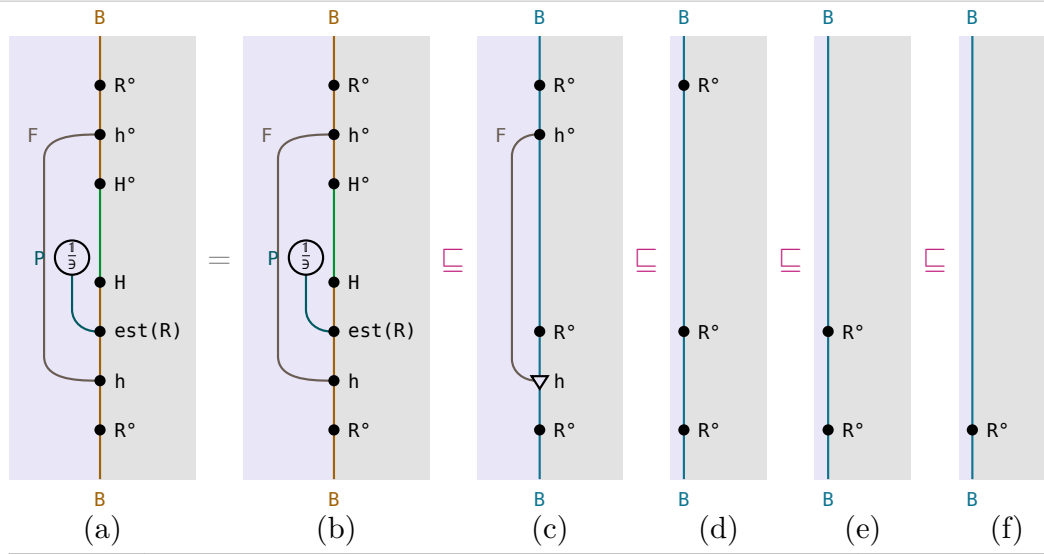


(a)	$H^\circ \frac{T^\circ}{\exists} \text{thin}(Q) \ P(F(\frac{H}{\exists} \text{est}(R)) \ h) \ \text{est}(R)$	$P(S) \ \text{est}(R) \sqsubseteq \epsilon \setminus (S \ R^\circ)$
(b)	$\sqsubseteq H^\circ \frac{T^\circ}{\exists} \text{thin}(Q) \ (\epsilon \setminus (F(\frac{H}{\exists} \text{est}(R)) \ h \ R^\circ))$	$H^\circ = h^\circ \ F(H^\circ) \ T$
(c)	$= h^\circ \ F(H^\circ) \ T \frac{T^\circ}{\exists} \text{thin}(Q) \ (\epsilon \setminus (F(\frac{H}{\exists} \text{est}(R)) \ h \ R^\circ))$	$T \frac{T^\circ}{\exists} \sqsubseteq \epsilon$
(d)	$\sqsubseteq h^\circ \ F(H^\circ) \ \epsilon \ \text{thin}(Q) \ (\epsilon \setminus (F(\frac{H}{\exists} \text{est}(R)) \ h \ R^\circ))$	$\epsilon \ \text{thin}(Q) \sqsubseteq Q^\circ \ \epsilon$



(a)	$\sqsubseteq h^\circ \ F(H^\circ) \ Q^\circ \ \epsilon \ (\epsilon \setminus (F(\frac{H}{\exists} \text{est}(R)) \ h \ R^\circ))$	$S \ (S \setminus R) \sqsubseteq R$
(b)	$\sqsubseteq h^\circ \ F(H^\circ) \ Q^\circ \ F(\frac{H}{\exists} \text{est}(R)) \ h \ R^\circ$	$\text{ThinCondition}(T, h, R, Q)$
(c)	$\sqsubseteq R^\circ \ h^\circ \ F(H^\circ) \ F(\frac{H}{\exists} \text{est}(R)) \ h \ R^\circ$	$\Rightarrow h^\circ \ F(H^\circ) \ Q^\circ \sqsubseteq R^\circ \ h^\circ \ F(H^\circ)$

1

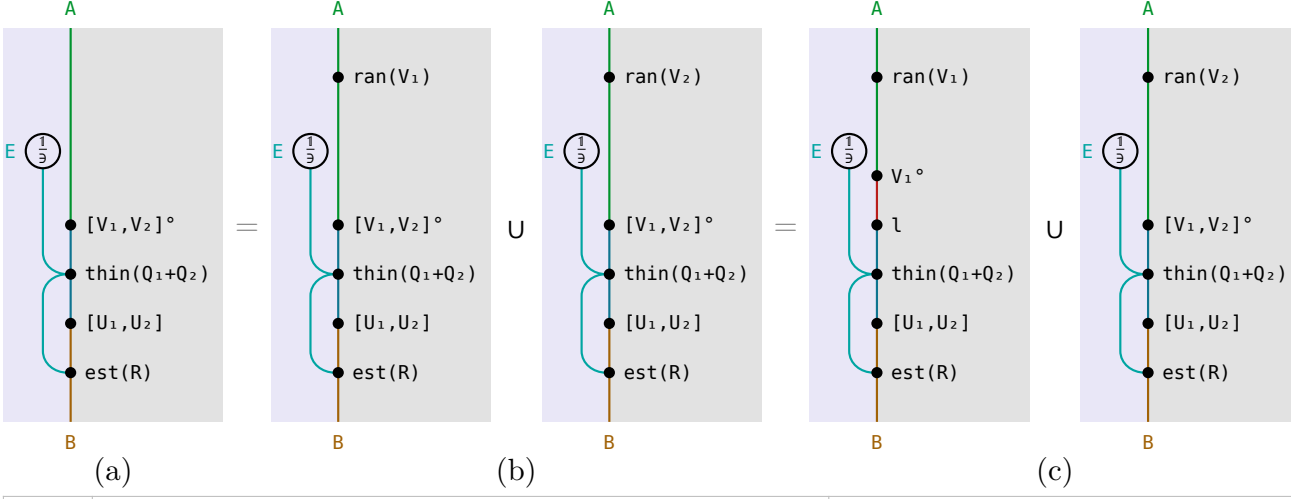


1 (a)	$R^\circ \ h^\circ \ F(H^\circ) \ F(\frac{H}{\exists} \ est(R)) \ h \ R^\circ$	
2 (b)	$= R^\circ \ h^\circ \ F(H^\circ \ \frac{H}{\exists} \ est(R)) \ h \ R^\circ$	$S^\circ \ \frac{S}{\exists} \ est(R) \subseteq R^\circ$
3 (c)	$\subseteq R^\circ \ h^\circ \ F(R^\circ) \ h \ R^\circ$	$\text{Map}(f)$ $\Rightarrow F(R) \ f \subseteq f \ R \Leftrightarrow f^\circ \ F(R) \ f \subseteq R$
4 (d)	$\subseteq R^\circ \ R^\circ \ R^\circ$	$R^\circ \ R^\circ \subseteq R^\circ$
5 (e)	$\subseteq R^\circ \ R^\circ$	$R^\circ \ R^\circ \subseteq R^\circ$
6 (f)	$\subseteq R^\circ$	

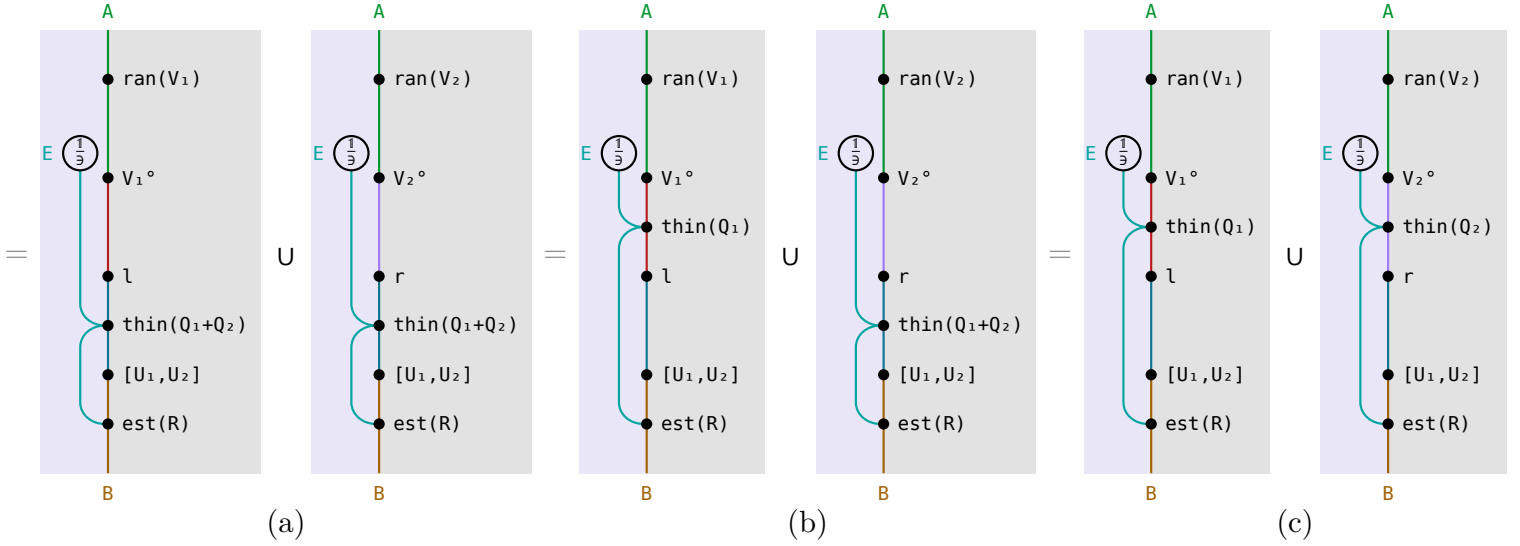
$$V_2 \ V_1^\circ = \emptyset \implies \frac{[V_1, V_2]^\circ}{3} \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ \text{est}(R) = (\text{ran}(V_1) \ \frac{V_1^\circ}{3} \text{thin}(Q_1) \ P(U_1) \ \text{est}(R)) \cup (\text{ran}(V_2) \ \frac{V_2^\circ}{3} \text{thin}(Q_2) \ P(U_2) \ \text{est}(R))$$

Proposition 9.1

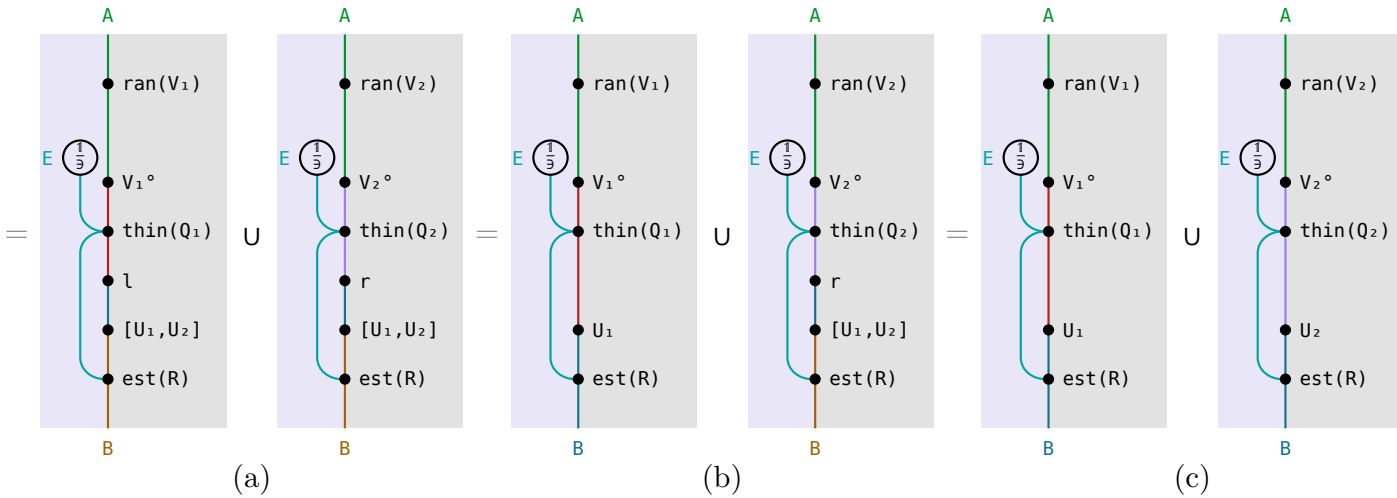
when V_1 and V_2 have disjoint ranges, the body over $[V_1, V_2]$ runs the V_1 problem on inputs V_1 reaches and the V_2 problem on inputs V_2 reaches



(a)	$\frac{[V_1, V_2]^\circ}{3} \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ \text{est}(R)$	$\frac{[V_1, V_2]^\circ}{3} \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ \text{est}(R) = (\text{ran}(V_1) \ \frac{[V_1, V_2]^\circ}{3} \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ \text{est}(R)) \cup (\text{ran}(V_2) \ \frac{[V_1, V_2]^\circ}{3} \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ \text{est}(R))$
(b)	$= (\text{ran}(V_1) \ \frac{[V_1, V_2]^\circ}{3} \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ \text{est}(R)) \cup (\text{ran}(V_2) \ \frac{[V_1, V_2]^\circ}{3} \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ \text{est}(R))$	$V_2 \ V_1^\circ = \emptyset \implies \text{ran}(V_1) \ \frac{[V_1, V_2]^\circ}{3} = \text{ran}(V_1) \ \frac{V_1^\circ}{3} \ P(l)$
(c)	$= (\text{ran}(V_1) \ \frac{V_1^\circ}{3} \ P(l) \ \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ \text{est}(R)) \cup (\text{ran}(V_2) \ \frac{[V_1, V_2]^\circ}{3} \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ \text{est}(R))$	$V_2 \ V_1^\circ = \emptyset \implies \text{ran}(V_2) \ \frac{[V_1, V_2]^\circ}{3} = \text{ran}(V_2) \ \frac{V_2^\circ}{3} \ P(r)$



(a)	$= (\text{ran}(V_1) \ \frac{V_1^\circ}{3} \ P(l) \ \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ \text{est}(R)) \cup (\text{ran}(V_2) \ \frac{V_2^\circ}{3} \ P(r) \ \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ \text{est}(R))$	$P(l) \ \text{thin}(Q_1 + Q_2) = \text{thin}(Q_1) \ P(l)$
(b)	$= (\text{ran}(V_1) \ \frac{V_1^\circ}{3} \ \text{thin}(Q_1) \ P(l) \ P([U_1, U_2]) \ \text{est}(R)) \cup (\text{ran}(V_2) \ \frac{V_2^\circ}{3} \ P(r) \ \text{thin}(Q_2) \ P([U_1, U_2]) \ \text{est}(R))$	$P(r) \ \text{thin}(Q_1 + Q_2) = \text{thin}(Q_2) \ P(r)$
(c)	$= (\text{ran}(V_1) \ \frac{V_1^\circ}{3} \ \text{thin}(Q_1) \ P(l) \ P([U_1, U_2]) \ \text{est}(R)) \cup (\text{ran}(V_2) \ \frac{V_2^\circ}{3} \ \text{thin}(Q_2) \ P(r) \ P([U_1, U_2]) \ \text{est}(R))$	



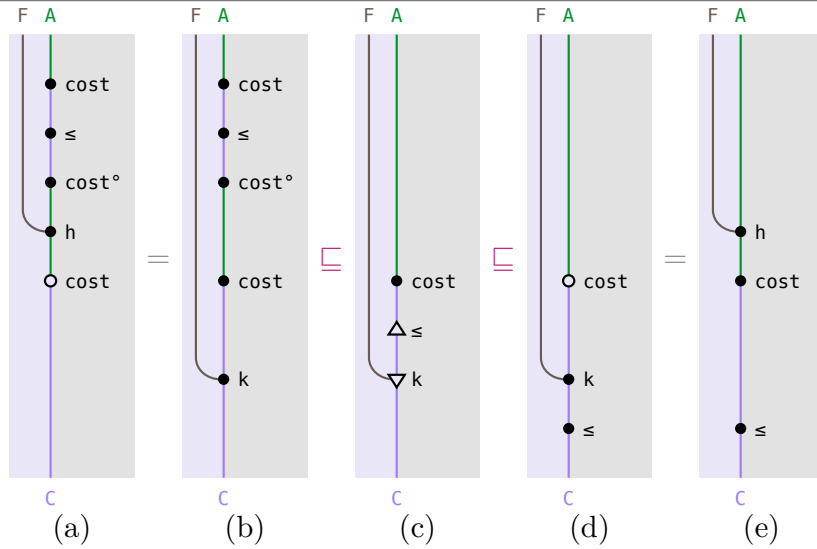
(a)	$= (\text{ran}(V_1) \ \frac{V_1^\circ}{3} \ \text{thin}(Q_1) \ P(l) \ P([U_1, U_2]) \ \text{est}(R)) \cup (\text{ran}(V_2) \ \frac{V_2^\circ}{3} \ \text{thin}(Q_2) \ P(r) \ P([U_1, U_2]) \ \text{est}(R))$	$l \ [R, S] = R$
(b)	$= (\text{ran}(V_1) \ \frac{V_1^\circ}{3} \ \text{thin}(Q_1) \ P(U_1) \ \text{est}(R)) \cup (\text{ran}(V_2) \ \frac{V_2^\circ}{3} \ \text{thin}(Q_2) \ P(r) \ P([U_1, U_2]) \ \text{est}(R))$	$r \ [R, S] = S$
(c)	$= (\text{ran}(V_1) \ \frac{V_1^\circ}{3} \ \text{thin}(Q_1) \ P(U_1) \ \text{est}(R)) \cup (\text{ran}(V_2) \ \frac{V_2^\circ}{3} \ \text{thin}(Q_2) \ P(U_2) \ \text{est}(R))$	

Map(cost) $\wedge$ h cost = F(cost) k $\wedge$ F($\leq$) k $\sqsubseteq$ k $\leq \implies$ F(cost $\leq$ cost $^\circ$) h $\sqsubseteq$ h cost $\leq$ cost $^\circ$ Proposition 9.2

if h then **cost** equals **F(cost)** then a **k** monotonic on $\leq$, then **h** is monotonic on **cost $\leq$ cost $^\circ$** ,
which compares two values by comparing their **costs** under $\leq$

Map(cost) $\wedge$ h cost = F(cost) k $\wedge$ F($\leq$) k $\sqsubseteq$ k $\leq \implies$ F(cost $\leq$ cost $^\circ$) h cost $\sqsubseteq$ h cost $\leq$

by shunting at the map **cost**, **F(cost $\leq$ cost $^\circ$)h $\sqsubseteq$ h cost $\leq$ cost $^\circ$** is **F(cost $\leq$ cost $^\circ$)h cost $\sqsubseteq$ h cost $\leq$** , which the chain proves



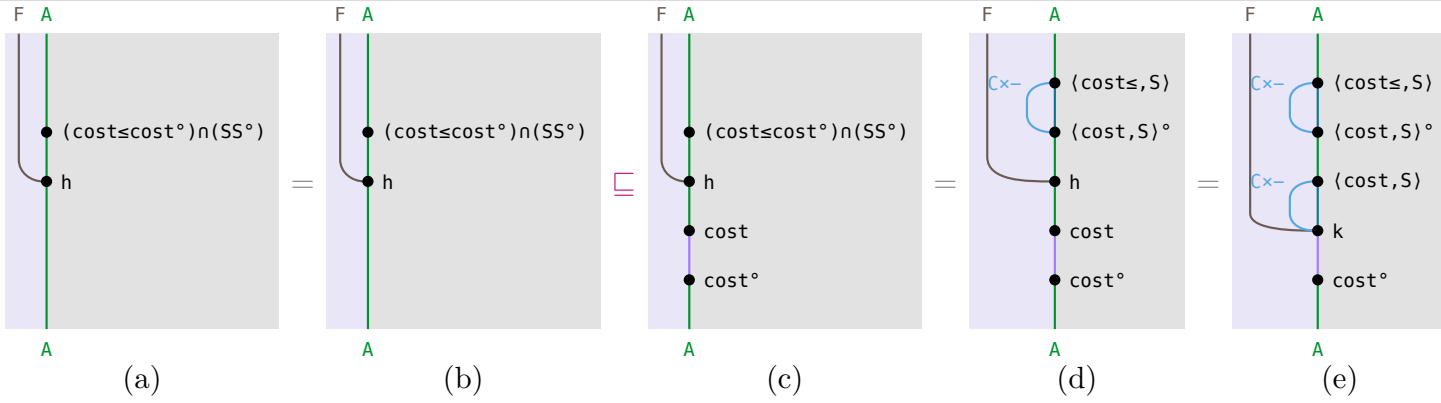
(a)	$F(\text{cost} \leq \text{cost}^\circ) \text{ h cost}$	$\text{h cost} = F(\text{cost}) \text{ k}$
(b)	$= F(\text{cost} \leq \text{cost}^\circ) F(\text{cost}) \text{ k}$	$\text{Map}(f) \wedge R = f \circ S \circ f^\circ$ $\implies R \circ f \sqsubseteq f \circ S$
(c)	$\sqsubseteq F(\text{cost}) F(\leq) \text{ k}$	$F(\leq) \text{ k} \sqsubseteq \text{k} \leq$
(d)	$\sqsubseteq F(\text{cost}) \text{ k} \leq$	$\text{h cost} = F(\text{cost}) \text{ k}$
(e)	$= \text{h cost} \leq$	

(9.1.1h)

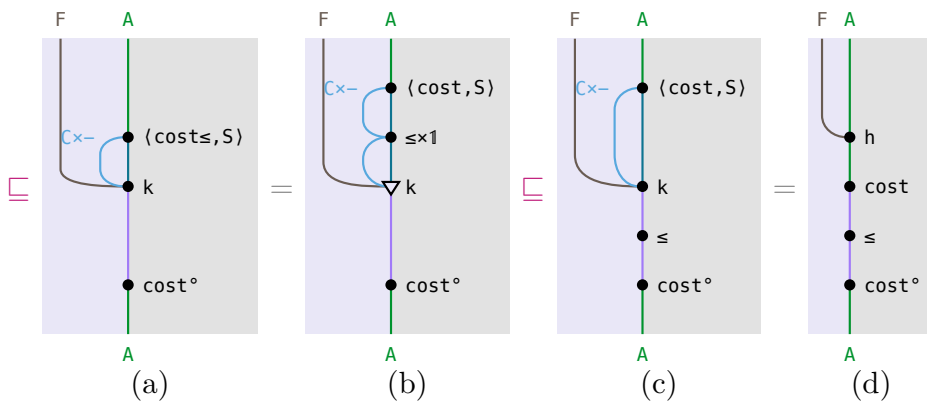
	the condition	how it is discharged
1	$F(R) \text{ h} \sqsubseteq \text{h} R$ Proposition 9.2, $R \triangleq \text{cost} \leq \text{cost}^\circ$, $\text{h cost} = F(\text{cost}) \text{ k}$, $F(\leq) \text{ k} \sqsubseteq \text{k} \leq$	monotonicity when the cost is itself a fold with a step k monotonic on $\leq$
2	$F(R \circ (H^\circ H)) \text{ h} \sqsubseteq \text{h} R$ Proposition 9.3, $R \triangleq \text{cost} \leq \text{cost}^\circ$, $\text{h cost} = F(\langle \text{cost}, H^\circ \rangle) \text{ k}$, $F(\leq \times 1) \text{ k} \sqsubseteq \text{k} \leq$, H° simple;	monotonicity in context : k may also read the input the part was built from
3	$QF(H) \text{ h} \sqsubseteq F(H) \text{ h} R$ at $Q \triangleq F(U, V)$ Proposition 9.4, U, V preorders, $F(U, R) \text{ h} \sqsubseteq \text{h} R$, $V H \sqsubseteq H R$	both conditions at once, split along the two arguments of a bifunctor

Map(cost) $\wedge$ Simple(S) $\wedge$ h cost = F($\langle$ cost,S $\rangle$) k $\wedge$ F($\leq \times 1$) k $\sqsubseteq$ k $\leq \Rightarrow$ F($\langle$ cost $\leq$ cost $^\circ$) $\cap$ (S S $^\circ$) h $\sqsubseteq$ h cost $\leq$ cost $^\circ$ Proposition 9.3

improving each part by **R** within its **S**-context, then assembling by **h**, is below **hR**



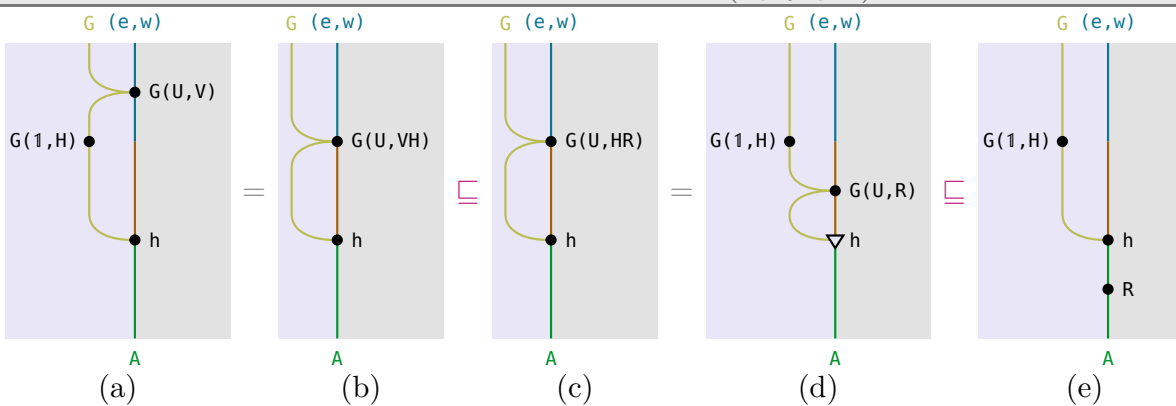
(a)	$F((\text{cost} \leq \text{cost}^\circ) \cap (S S^\circ)) h$	
(b)	$= F((\text{cost} \leq \text{cost}^\circ) \cap (S S^\circ)) h$	$\text{Map}(f)$ $\Rightarrow 1 \sqsubseteq f f^\circ$
(c)	$\sqsubseteq F((\text{cost} \leq \text{cost}^\circ) \cap (S S^\circ)) h \text{ cost cost}^\circ$	$\langle X, Y \rangle \langle R, S \rangle^\circ = (X R^\circ) \cap (Y S^\circ)$
(d)	$= F(\langle \text{cost} \leq, S \rangle \langle \text{cost}, S \rangle^\circ) h \text{ cost cost}^\circ$	$h \text{ cost} = F(\langle \text{cost}, S \rangle) k$
(e)	$= F(\langle \text{cost} \leq, S \rangle \langle \text{cost}, S \rangle^\circ \langle \text{cost}, S \rangle) k \text{ cost}^\circ$	



(a)	$\sqsubseteq F(\langle \text{cost} \leq, S \rangle) k \text{ cost}^\circ$	$\langle X, Y \rangle (R \times 1) = \langle X R, Y \rangle$
(b)	$= F(\langle \text{cost}, S \rangle) F(\leq \times 1) k \text{ cost}^\circ$	$F(\leq \times 1) k \sqsubseteq k \leq$
(c)	$\sqsubseteq F(\langle \text{cost}, S \rangle) k \leq \text{cost}^\circ$	$h \text{ cost} = F(\langle \text{cost}, S \rangle) k$
(d)	$= h \text{ cost} \leq \text{cost}^\circ$	

G(U,R) h $\sqsubseteq$ h R $\wedge$ V H $\sqsubseteq$ H R $\Rightarrow$ G(U,V) G(1,H) h $\sqsubseteq$ G(1,H) h R Proposition 9.4

thinning the parts by **U** in the first argument and by **V** in the second (**G(U,V)**), then solving by **H** and assembling by **h**, gives only what solving and assembling and then improving by **R** gives (**G(1,H)hR**)

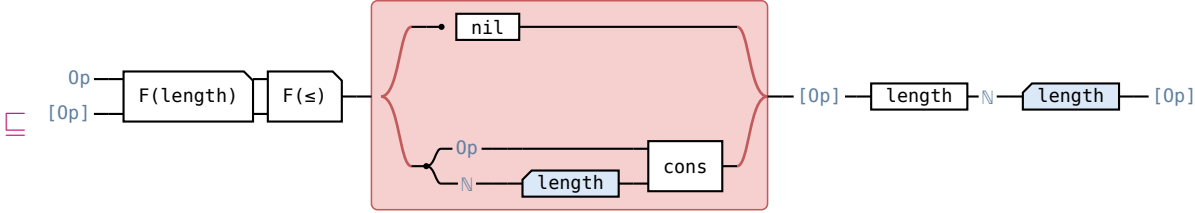
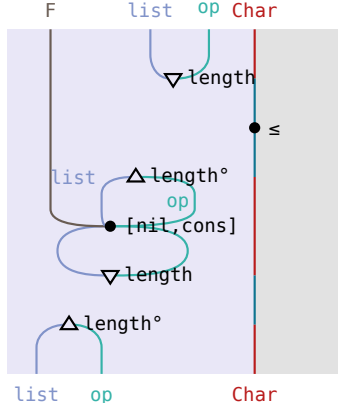
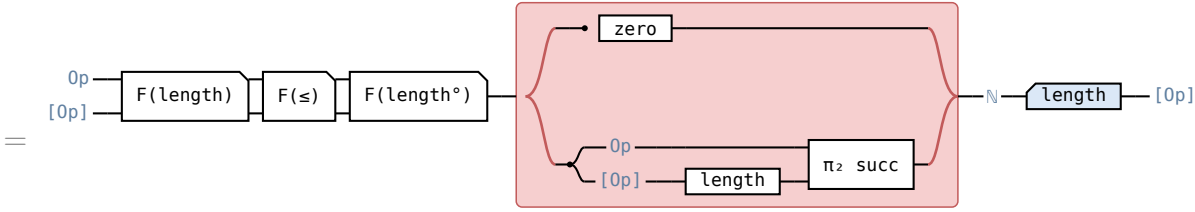
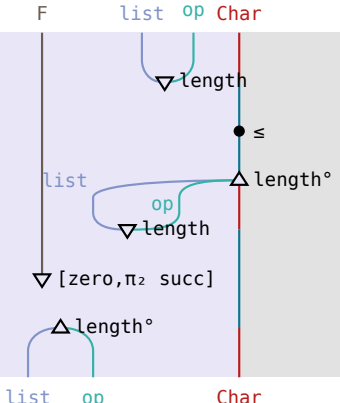
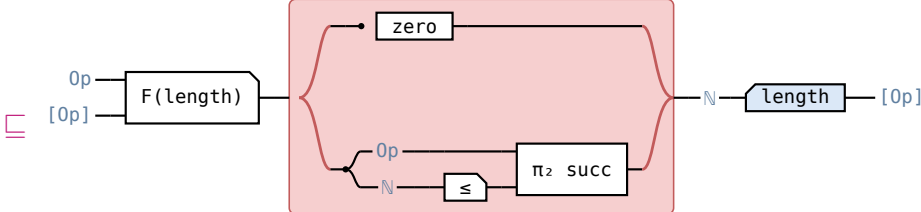
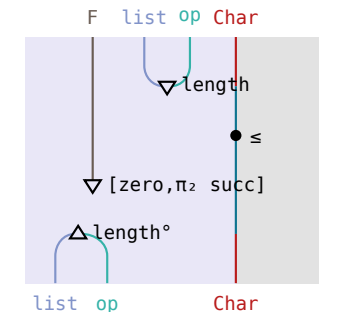
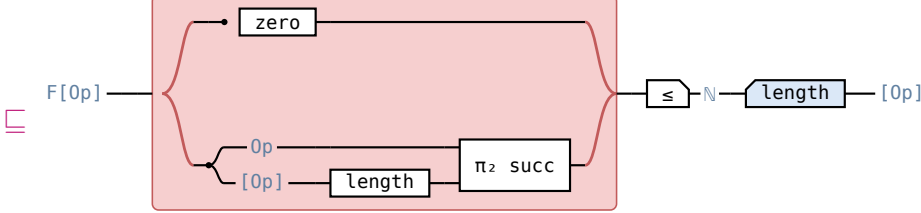
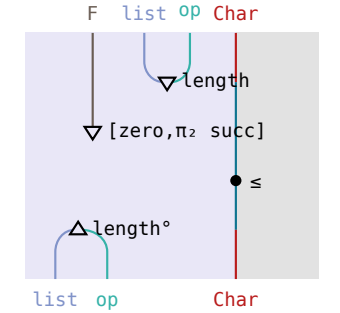
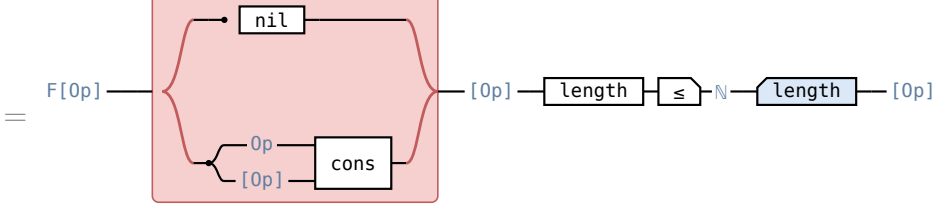
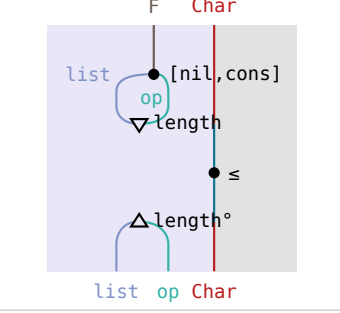
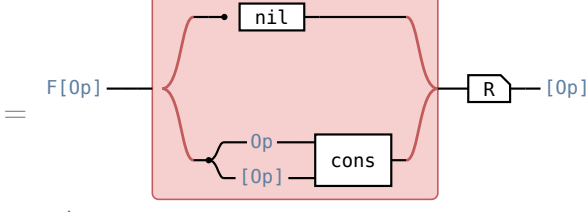
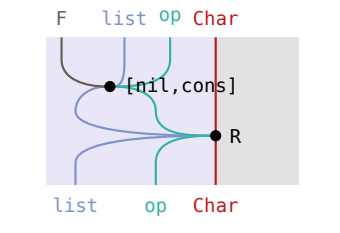


(a)	$G(U,V) G(1,H) h$	
(b)	$= G(U,VH) h$	$V H \sqsubseteq H R$
(c)	$\sqsubseteq G(U,HR) h$	
(d)	$= G(1,H) G(U,R) h$	$G(U,R) h \sqsubseteq h R$
(e)	$\sqsubseteq G(1,H) h R$	

§9.2. The string edit problem

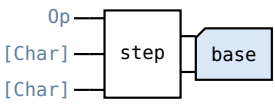
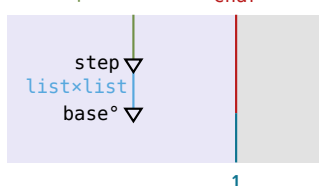
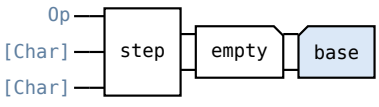
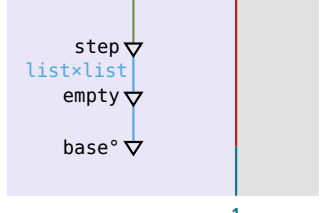
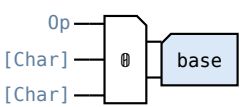
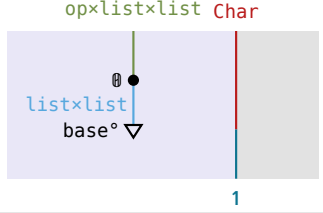
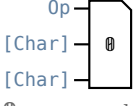
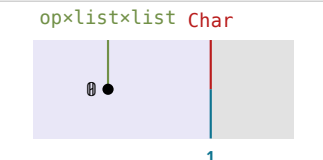
(9.2a)

	name	type	definition	meaning
1	Op	Type	$(\exists a. o = \text{cpy}(a)) \vee (\exists a. o = \text{del}(a)) \vee (\exists a. o = \text{ins}(a))$	an edit operation copies, deletes or inserts one character
2			$\text{FX} = 1 + (\text{Op} \times \text{X})$	$\text{F}(\text{Op}, \text{X})$ is the empty sequence or one operation paired with an X
3	edit	$[\text{Op}] \rightarrow [\text{Char}] \times [\text{Char}]$	$\text{edit} = ([\text{base}, \text{step}])$, $\text{edit}([]) = ([], [])$, $\text{edit}([\text{op}] \# \text{es}) = \text{step}(r((\text{op}, \text{edit}(\text{es}))))$	the two strings an edit sequence turns one into the other
4			$\text{length} = ([\text{zero}, \pi_2 \text{ succ}])$	the number of operations in a sequence
5	R	$[\text{Op}] \rightarrow [\text{Op}]$	$\text{R} = \text{length} \leq \text{length}^\circ$, $\text{R}(\text{es}, \text{fs}) \iff \text{length}(\text{es}) \leq \text{length}(\text{fs})$	es is no longer than fs
6	V	$[\text{Char}] \times [\text{Char}] \rightarrow [\text{Char}] \times [\text{Char}]$	$\text{V} = \text{suffix}^\circ \times \text{suffix}^\circ$, $\text{V}(\text{p}, \text{q}) \iff \text{suffix}(\pi_1(\text{p}), \pi_1(\text{q})) \wedge \text{suffix}(\pi_2(\text{p}), \pi_2(\text{q}))$	each string of the first pair is a suffix of the matching string of the second
7	Q	$\text{F}([\text{Char}] \times [\text{Char}]) \rightarrow \text{F}([\text{Char}] \times [\text{Char}])$	$\text{Q}(\text{l}(\text{val}), \text{l}(\text{val}')) = \text{true}$, $\text{Q}(\text{r}(\text{p}), \text{r}(\text{q})) = \text{V}(\pi_2(\text{p}), \pi_2(\text{q}))$, $(\forall \text{val}. \forall \text{val}'. u = \text{l}(\text{val}) \implies v = \text{l}(\text{val}') \implies \text{false}) \wedge (\forall \text{p}. \forall \text{q}. u = \text{r}(\text{p}) \implies v = \text{r}(\text{q}) \implies \text{false}) \implies \text{Q}(u, v) = \text{false}$, $\text{Q}(\text{l}(u), \text{l}(u))$, $\text{Q}(\text{r}(\text{p}), \text{r}(\text{q})) \iff \text{V}(\pi_2(\text{p}), \pi_2(\text{q}))$	the identity on the base summand; on the step summand any two operations, and the two pairs of strings by V
8	empty	$[\text{Char}] \times [\text{Char}] \rightarrow [\text{Char}] \times [\text{Char}]$	$\text{empty}(\text{p}, \text{q}) \iff \text{p} = \text{q} \wedge \text{p} = ([], [])$	holds only at the pair of two empty strings
9	unstep	$[\text{Char}] \times [\text{Char}] \rightarrow [\text{Op}] \times [\text{Char}] \times [\text{Char}]$	$\text{q} \in \text{unstep}(\text{p}) \implies \text{step}(\text{r}(\text{q})) = \text{p}$, $\text{unstep}([], []) = []$, $\text{unstep}([[\text{a}] \# \text{xs}, []]) = [(\text{del}(\text{a}), (\text{xs}, []))]$, $\text{unstep}([[], [\text{b}] \# \text{ys}]) = [(\text{ins}(\text{b}), ([], \text{ys}))]$, $\text{unstep}([[\text{a}] \# \text{xs}, [\text{b}] \# \text{ys}]) = \text{if } \text{a} = \text{b} \text{ then } [(\text{cpy}, \text{xs}, \text{ys})] \text{ else } [(\text{del}, \text{xs}, [\text{b}] \# \text{ys}), (\text{ins}, [\text{a}] \# \text{xs}, \text{ys})]$	every split of a pair of strings into one operation and a smaller pair; step puts it back
10	[nil, cons]	$\text{L} + \text{E} \times \mu\text{F} \rightarrow \mu\text{F}$	$[\text{nil}, \text{cons}](\text{l}(u)) = []$, $[\text{nil}, \text{cons}](\text{r}((\text{o}, \text{es}))) = [\text{o}] \# \text{es}$	$\alpha = [\text{nil}, \text{cons}]$: the empty edit sequence, or one operation in front of a sequence
11	step	$\text{Unit} + \text{Op} \times [\text{Char}] \times [\text{Char}] \rightarrow [\text{Char}] \times [\text{Char}]$	$\text{step}(\text{l}(u)) = ([], [])$, $\text{step}(\text{r}((\text{cpy}(\text{a}), (\text{xs}, \text{ys})))) = ([\text{a}] \# \text{xs}, [\text{a}] \# \text{ys})$, $\text{step}(\text{r}((\text{del}(\text{a}), (\text{xs}, \text{ys})))) = ([\text{a}] \# \text{xs}, \text{ys})$, $\text{step}(\text{r}((\text{ins}(\text{a}), (\text{xs}, \text{ys})))) = (\text{xs}, [\text{a}] \# \text{ys})$	[base, step] : the two empty strings, or one operation added to the front of the pair

circuit	Hinze–Marsden
 length is a map, so entire: $1 \sqsubseteq \text{length} \text{ length}^\circ$	
 $\alpha \text{ length} = F(\text{length})[\text{zero}, \pi_2 \text{ succ}] : \text{length} \triangleq ([\text{zero}, \pi_2 \text{ succ}]) - \text{es R fs} \iff \text{length}(\text{es}) \leq \text{length}(\text{fs})$	
 length is a map, so simple, and F is monotonic: $F(\text{length}^\circ)F(\text{length}) \sqsubseteq F(1) = 1$	
 succ is monotonic on $\leq$: $F(\leq)[\text{zero}, \pi_2 \text{ succ}] \sqsubseteq [\text{zero}, \pi_2 \text{ succ}] \leq$	
 $F(\text{length})[\text{zero}, \pi_2 \text{ succ}] = \alpha \text{ length}$ again	
 $R \triangleq \text{length} \leq \text{length}^\circ - \text{es R fs} \iff \text{length}(\text{es}) \leq \text{length}(\text{fs})$	

$Q \vdash F(\text{edit}^\circ) [\text{nil}, \text{cons}] \sqsubseteq F(\text{edit}^\circ) [\text{nil}, \text{cons}] R$ (9.2d) every edit sequence built from a decomposition Q puts above a given one — any operation, each string lengthened at the front — is at least as long as one built from the given one	
circuit	Hinze–Marsden
$=$ $Q \triangleq 1 + (U \times V)$ is $F(U, V)$ at $U \triangleq \tau \mid Q(l(\text{val}), l(\text{val}')) = \text{true}, Q(r(p), r(q)) = V(\pi_2(p), \pi_2(q)), (\forall \text{val}. \forall \text{val}'. u = l(\text{val}) \Rightarrow v = l(\text{val}') \Rightarrow \text{false}) \wedge (\forall p. \forall q. u = r(p) \Rightarrow v = r(q) \Rightarrow \text{false})$ $\Rightarrow Q(u, v) = \text{false}$; and the bifunctor F preserves composition: $F(U, V)F(1, \text{edit}^\circ) = F(U, V \text{edit}^\circ)$	
$\sqsubseteq$ Proposition 9.4's second condition $V \text{edit}^\circ \sqsubseteq \text{edit}^\circ R \mid \text{edit} V^\circ \sqsubseteq R^\circ \text{edit}$	
$=$ F preserves composition: $F(U, \text{edit}^\circ R) = F(1, \text{edit}^\circ)F(U, R)$	
$\sqsubseteq$ Proposition 9.4's first condition $F(\tau, R) \triangleq \alpha R$, left as an exercise in the book: cons adds one to both lengths whatever the two operations are	

$\text{edit } V^\circ \sqsubseteq R^\circ \text{ edit}$ (9.2e) cutting a front off either string an edit sequence produces ($\text{edit } V^\circ$) leaves a pair that a sequence no longer produces ($R^\circ \text{ edit}$)	
circuit	Hinze–Marsden
$=$ $V \triangleq \text{suffix}^\circ \times \text{suffix}^\circ$, and $\times$ preserves composition	
$\sqsubseteq$ $\text{edit } (\text{suffix} \times 1) \sqsubseteq R^\circ \text{ edit}$: drop the operation that produced the head, or weaken its cpy to an ins. Proved by induction at suffix directly, not through $\text{suffix} = \text{tail}^*$ and $BA \sqsubseteq CB \implies BA^* \sqsubseteq C^*B$ as the book does	
$\sqsubseteq$ $\text{edit } (1 \times \text{suffix}) \sqsubseteq R^\circ \text{ edit}$, the mirror image: cpy weakens to a del	
$\sqsubseteq$ R° is transitive, $\leq$ being so	

$\text{step } \text{base}^\circ = \emptyset$ (9.2f) no pair of strings is both a result of step and the result $([], [])$ of base	
circuit	Hinze–Marsden
	
 base returns only $([], [])$, so $\text{base}^\circ = \text{empty } \text{base}^\circ$	
 $\text{step } \text{empty} = \emptyset$: cpy and del put a character on the left string, ins one on the right	
 $\emptyset$ composed with anything is $\emptyset$	

1	$\text{mle}(xs, ys) = \text{head}(\text{column}(xs, ys))$, $\text{column}(xs, ys) = [\text{mle}(u, ys) \mid u \leftarrow \text{tails}(xs)]$ $= \text{column}(xs) = ([\text{fstcol}(xs), \text{nextcol}(xs)] \mid \text{fstcol} = \text{list}(\text{del}) \text{ tails}$ the tabulation: $\text{mle}(xs, ys)$ needs $\text{mle}(u, v)$ for every tail u of xs and v of ys , so the columns are built right to left
2	$\text{column}(xs, [b] \# ys) = \text{nextcol}(xs, (b, \text{column}(xs, ys)))$ $= \text{nextcol}(xs)(b, us) = ([\text{base}(b, \text{last}(us)), \text{step}(b)] \mid xus), \text{ xus} = \text{zip}(xs, \text{zip}(\text{init}(us), \text{tail}(us)))$ each column is a fold built bottom to top, over xs zipped with the adjacent pairs of the column to its right
3	$\text{base}(b, u) = [[\text{ins}(b)] \# u]$ $= \text{step}(b)((a, (u, v)), ws) = (a \rightarrow b \rightarrow [[\text{cpy}(a)] \# v] \# ws, [\text{bmin}(R)([\text{del}(a)] \# w, [\text{ins}(b)] \# u)] \# ws)$ $w = \text{head}(ws)$; these base , step are not edit 's. An entry depends on the one below it (a delete), the one to its right (an insert), and the one below that (a copy) — quadratic in the two lengths

§9.3. Optimal bracketing

(9.3a)

	name	type	definition	meaning
1	tree(A)	Type	$\text{tree}(A) \triangleq \text{tip}(A) \mid \text{bin}(\text{tree}(A), \text{tree}(A))$	a tree is a tip holding one value, or a node joining two trees; $h \triangleq [\text{tip}, \text{bin}]$
2	flatten	$\text{tree } A \rightarrow L A$	$\langle [\text{wrap}, \text{cat}] \rangle = \text{flatten}, \text{flatten}(\text{tip}(a)) = [a], \text{flatten}(\text{bin}(l, r)) = \text{cat}(\text{flatten}(l), \text{flatten}(r))$	the tips of a tree, left to right; $H = \text{flatten}^\circ$ builds every tree over a list
3	opb(p)	$\text{Int} \times S$	$\text{opb}(p) \triangleq ((\text{cb}(\pi_2(\pi_1(p)), \pi_2(\pi_2(p))) + \pi_1(\pi_1(p))) + \pi_1(\pi_2(p)), \text{sb}(\pi_2(\pi_1(p)), \pi_2(\pi_2(p))))$	joining two subtrees costs cb of their sizes plus the cost of each
4	$\langle \text{cost}, \text{size} \rangle$	$\text{tree } A \rightarrow \text{Int} \times S$	$\langle \text{cost}, \text{size} \rangle(\text{tip}(a)) = (0, \text{st}(a)), \langle \text{cost}, \text{size} \rangle(\text{bin}(l, r)) = \text{opb}((\langle \text{cost}, \text{size} \rangle(l), \langle \text{cost}, \text{size} \rangle(r)))$	the cost of evaluating a bracketing, paired with the size of its result
5	sz	$L A \rightarrow S$	$\text{sz}([a]) = \text{st}(a), \text{sz}([a]\#x) = \text{sb}(\text{st}(a), \text{sz}(x)), \text{Assoc}(\text{sb}) \Rightarrow \pi_2(\langle \text{cost}, \text{size} \rangle(t)) = \text{sz}(\text{flatten}(t))$	sb associative, so the size depends only on the flattened list
6	R	$\text{tree}(A) \rightarrow \text{tree}(A)$	$R = \text{cost} \leq \text{cost}^\circ, R(t, t') \Leftrightarrow \text{cost}(t) \leq \text{cost}(t')$	t costs no more than t'
7	g	$F(\text{Int} \times L A) \rightarrow \text{Int}$	$g \triangleq [\neg 0, (1 \times \text{sz})^2 \text{ opb } \pi_1], g(l(a)) = 0, g(r(((cx, x), cy, y))) = (\text{cb}(\text{sz}(x), \text{sz}(y)) + cx) + cy$	the cost of the top step alone: zero at a tip, cb of the two sizes at a node
8	nonsingle	$L A \rightarrow L A$	$x \text{ nonsingle } y \Leftrightarrow x = y \wedge (\exists a. \exists z. x = [a]\#z)$	holds at the lists of two or more elements
9	splits	$L A \rightarrow [L A \times L A]$	$\text{splits} = \langle \text{inits}^+, \text{tails}^+ \rangle \text{ zip}, \text{splits}(x) = \text{zip}(((\text{inits}^+)(x), (\text{tails}^+)(x)))$	every way to cut a list into two non-empty parts — an implementation of $\frac{\text{cat}^\circ}{\exists}$
10	array	$L A \rightarrow [[\text{tree } A]]$	$\text{array} \triangleq \text{inits list}(\text{row}), \text{array}(x, \text{xss}) \Leftrightarrow \text{xss} = \text{list}(\text{row}, \text{inits}(x))$	the best trees of every segment, held as rows
11	row	$L A \rightarrow [\text{tree } A]$	$\text{row} = \text{tails list}(\text{mct}), \text{row}(x, \text{ys}) \Leftrightarrow \text{ys} = \text{list}(\text{mct}, \text{tails}(x))$	the best tree of every non-empty suffix
12	col	$L A \rightarrow [\text{tree } A]$	$\text{col} = \text{inits list}(\text{mct}), \text{col}(x, \text{ys}) \Leftrightarrow \text{ys} = \text{list}(\text{mct}, \text{inits}(x))$	the best tree of every non-empty prefix
13	mix	$[\text{tree } A] \times [\text{tree } A] \rightarrow \text{tree}(A)$	$\text{mix} = \text{zip list}(\text{bin}) \text{ minlist}(R), \text{mix}(p, t) \Leftrightarrow t = \text{minlist}(R) (\text{list bin zip})$	join the matching prefix and suffix trees and keep a cheapest
14	next	$[\text{tree } A] \times [\text{tree } A] \rightarrow [\text{tree } A]$	$\text{next} = \langle \pi_1, \text{mix} \rangle \text{ snoc}, \text{next}(p, \text{zs}) \Leftrightarrow \text{zs} = \text{snoc}((\pi_1(p), \text{minlist}(R) (\text{list bin zip})))$	add the next best tree to the end of the column
15	process	$A \times [[\text{tree } A]] \rightarrow [\text{tree } A]$	$\text{process} \triangleq ((\text{tip wrap}) \times 1) \text{ loop}(\text{next}), \text{process}(a, \text{xss}, \text{ys}) \Leftrightarrow \text{loop}$	start the column at a tip and run next over the rows

$F(\leq \times 1) \ g \sqsubseteq g \leq$ (9.6)	
raising the cost of either subtree ($\leq \times 1$ in both slots) and then applying g returns a value at least what g returns on the costs before raising ($g \leq$)	
circuit	Hinze–Marsden
$F(\text{Int} \times \text{LA}) \rightarrow \boxed{F(\leq \times 1)} \rightarrow \boxed{g} \rightarrow \text{Int}$	
$A + \text{Int} \times \text{LA} \times \text{Int} \times \text{LA} \rightarrow$ $=$ definition of $g \rightarrow g \triangleq [\neg 0, (1 \times \text{sz})^2 \text{ opb } \pi_1]$	
$A + \text{Int} \times \text{LA} \times \text{Int} \times \text{LA} \rightarrow$ $\sqsubseteq$ definition of opb , and $+$ monotonic	
$A + \text{Int} \times \text{LA} \times \text{Int} \times \text{LA} \rightarrow$ $\sqsubseteq$ $\leq$ reflexive, so $\text{zero} \sqsubseteq \text{zero} \leq$; coproducts	
$F(\text{Int} \times \text{LA}) \rightarrow \boxed{g} \rightarrow \boxed{\leq} \rightarrow \text{Int}$ $=$ definition of $g \rightarrow g \triangleq [\neg 0, (1 \times \text{sz})^2 \text{ opb } \pi_1]$	

$$\text{mct}=(\text{single}\rightarrow\text{head tip},(\text{init col},\text{tail row})\text{ mix}), \text{ Assoc}(\text{sb}) \Rightarrow (\mu X : \frac{[\text{wrap},\text{cat}]^\circ}{3} \text{P}([\text{tip},(X \times X) \text{ bin}]) \text{ est}(\text{R})) \sqsubseteq \frac{\text{flatten}^\circ}{3} \text{est}(\text{R}) \quad (9.3b)$$

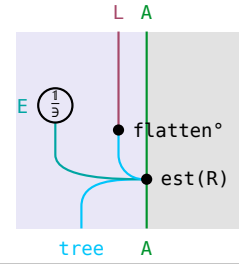
a least-cost bracketing of $a_1 \oplus \dots \oplus a_n$ is read off an array holding one best tree per non-empty segment, each entry built from the column to its left and the row below it

circuit

$$\text{LA} \rightarrow \frac{1}{3} \rightarrow \text{P}(\text{LA}) \rightarrow \text{E}(\text{flatten}^\circ) \rightarrow \text{P}(\text{tree}(\text{A})) \rightarrow \text{est}(\text{R}) \rightarrow \text{tree}(\text{A})$$

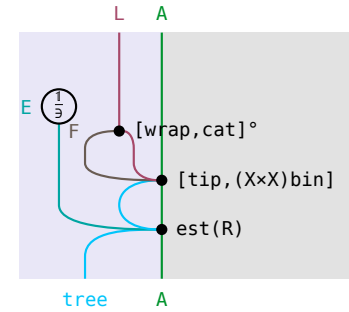
the specification — $\text{flatten}(\text{tip}(\text{a})) = [\text{a}]$, $\text{flatten}(\text{bin}(\text{l},\text{r})) = \text{cat}(\text{flatten}(\text{l}),\text{flatten}(\text{r}))$

Hinze–Marsden



$$\text{LA} \rightarrow \frac{1}{3} \rightarrow \text{P}(\text{LA}) \rightarrow \text{E}([\text{wrap},\text{cat}]^\circ) \rightarrow \text{PF}(\text{LA}) \rightarrow \text{P}([\text{tip},(X \times X) \text{ bin}]) \rightarrow \text{P}(\text{tree}(\text{A})) \rightarrow \text{est}(\text{R}) \rightarrow \text{tree}(\text{A})$$

split the list in every way, bracket both halves, join. The condition is monotonicity **in context**, $\sqsubseteq \text{F}(\text{Rn}(\text{flatten} \text{ flatten}^\circ)) \text{ h} \sqsubseteq \text{hR}$ — only trees with the same flattening are compared — which is Proposition 9.3 at $\text{H}^\circ = \text{flatten}$, a map, with (9.5) $[\text{tip},\text{bin}] \text{ cost} = (1 + (\text{cost}, \text{flatten})^2) \text{g}$ (the cost of a node reads only the cost and the flattening of its two subtrees) and (9.6) $(1 + (\leq \times 1)^2) \text{g} \sqsubseteq \text{g} \leq \text{g}$ (g monotonic on $\leq$ in its two cost arguments)

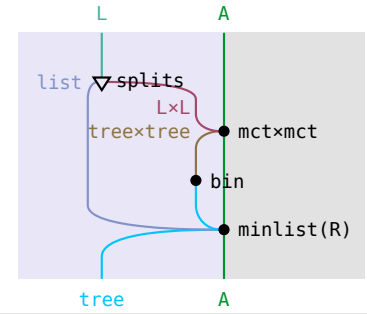


$$\text{LA} \rightarrow \frac{1}{3} \rightarrow \text{P}(\text{LA}) \rightarrow \text{E}(\text{cat}^\circ) \rightarrow \text{P}(\text{LA}^2) \rightarrow \text{P}((X \times X) \text{ bin}) \rightarrow \text{P}(\text{tree}(\text{A})) \rightarrow \text{est}(\text{R}) \rightarrow \text{tree}(\text{A})$$

= Proposition 9.1: **wrap** and **cat** have disjoint ranges, and **single** is the coreflexive on singleton lists, where **wrap** returns

$$\text{LA} \rightarrow \text{splits} \rightarrow [\text{LA}^2] \rightarrow \text{list}((\text{mct} \times \text{mct}) \text{ bin}) \rightarrow [\text{tree}(\text{A})] \rightarrow \text{minlist}(\text{R}) \rightarrow \text{tree}(\text{A})$$

$\sqsubseteq \text{splits} \triangleq \langle \text{inits}^+, \text{tails}^+ \rangle$ zip implements $\frac{\text{cat}^\circ}{3}$ and $\text{minlist}(\text{R})$ implements $\text{est}(\text{R})$. Exponential, since the segments of one list overlap



$$\text{mct}=(\text{single}\rightarrow\text{head tip},(\text{init col},\text{tail row})\text{ mix}) \quad (9.7)$$

= the tabulation: $\text{mct } \text{xs}$ is needed for every non-empty segment xs , so the values are held as an array of rows, $\text{array} \triangleq \text{inits list}(\text{row})$, $\text{row} \triangleq \text{tails list}(\text{mct})$, $\text{col} \triangleq \text{inits list}(\text{mct})$, $\text{mix} \triangleq \text{zip list}(\text{bin}) \text{ minlist}(\text{R})$

$$\text{col}=(\text{single}\rightarrow\text{head tip wrap},(\text{init col},\text{tail row})\text{ next}) \quad (9.8)$$

$$\text{cons col}=(1 \times \text{array}) \text{ process} \quad (9.9)$$

$$\text{row}=(\text{single}\rightarrow\text{head tip wrap},(\text{mct},\text{tail row})\text{ cons}) \quad (9.10)$$

a column extends the column to its left, a row the row below it; (9.9) is (9.8) rewritten as a loop, $\text{next} \triangleq \langle \pi_1, \text{mix} \rangle \text{ snoc}$, $\text{process} \triangleq ((\text{tip wrap}) \times 1) \text{ loop}(\text{next})$

$$\text{array}=(\text{fstcol},\text{addcol}), \text{ fstcol} \triangleq \text{tip wrap wrap}$$

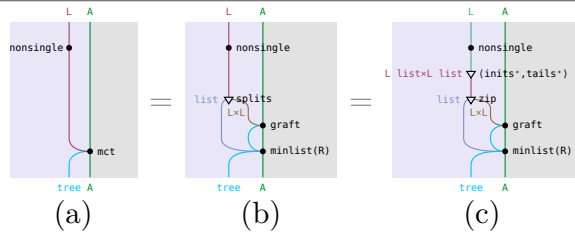
$$\text{addcol} \triangleq \langle \pi_1 \text{ tip wrap}, \text{step} \rangle \text{ cons}, \text{ step} \triangleq (\text{process tail}, \pi_2) \text{ zip list}(\text{cons})$$

the program: one fold building the array column by column, cubic in the length of the input

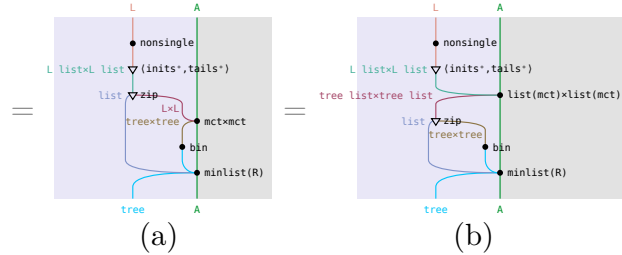
nonsingle mct = nonsingle (init col,tail row) mix

(9.7)

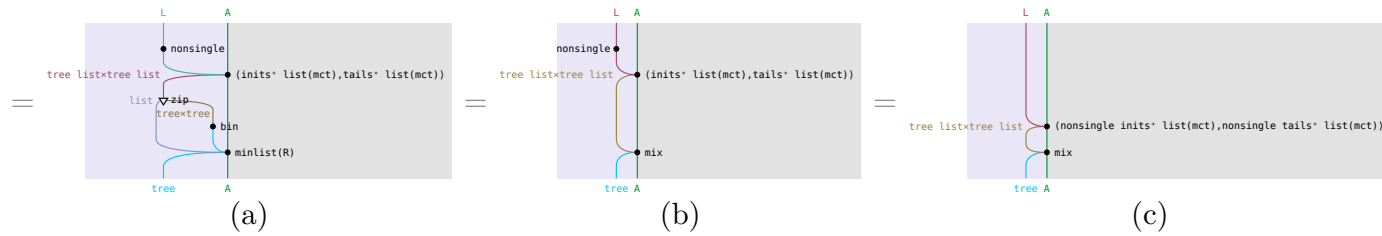
on a list of two or more elements, **mct** gives what **mix** gives on the column of its **init** and the row of its **tail**



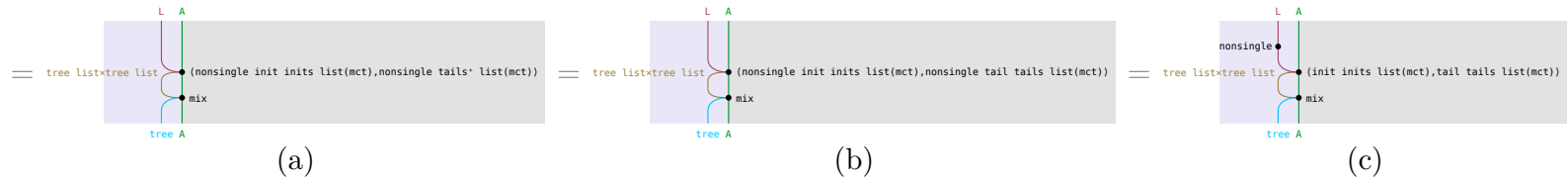
(a)	nonsingle mct	nonsingle mct = nonsingle splits list(graft) minlist(R)
(b)	= nonsingle splits list(graft) minlist(R)	splits = (inits+,tails+) zip
(c)	= nonsingle (inits+,tails+) zip list(graft) minlist(R)	graft $\triangleq$ (mct $\times$ mct) bin



(a)	= nonsingle (inits+,tails+) zip list(mct $\times$ mct) list(bin) minlist(R)	zip list(f $\times$ g) M = (list(f) $\times$ list(g)) zip M
(b)	= nonsingle (inits+,tails+) (list(mct) $\times$ list(mct)) zip list(bin) minlist(R)	(R,S) (U $\times$ V) = (R U,S V)



(a)	= nonsingle (inits+ list(mct),tails+ list(mct)) zip list(bin) minlist(R)	mix = zip list(bin) minlist(R)
(b)	= nonsingle (inits+ list(mct),tails+ list(mct)) mix	nonsingle (U,V) = (nonsingle U,nonsingle V)
(c)	= (nonsingle inits+ list(mct),nonsingle tails+ list(mct)) mix	nonsingle inits+ M = nonsingle init inits M



(a)	= (nonsingle init inits list(mct),nonsingle tails+ list(mct)) mix	nonsingle tails+ M = nonsingle tail tails M
(b)	= (nonsingle init inits list(mct),nonsingle tail tails list(mct)) mix	nonsingle (U,V) = (nonsingle U,nonsingle V)
(c)	= nonsingle (init inits list(mct),tail tails list(mct)) mix	col = inits list(mct)

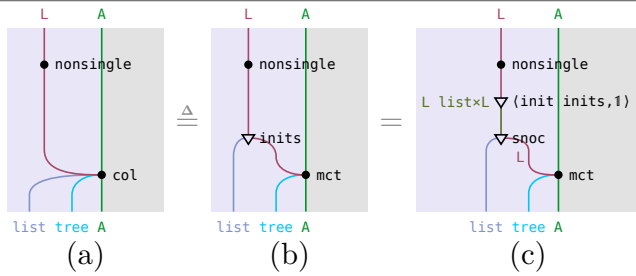


(a)	= nonsingle (init col,tail tails list(mct)) mix	row = tails list(mct)
(b)	= nonsingle (init col,tail row) mix	

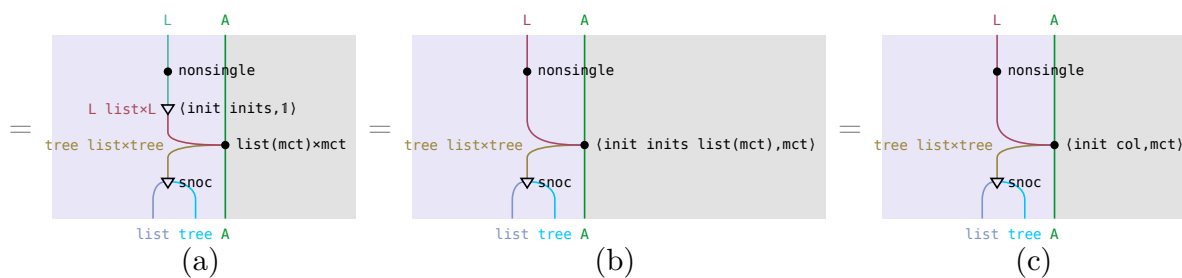
nonsingle col = nonsingle (init col,tail row) next

(9.8)

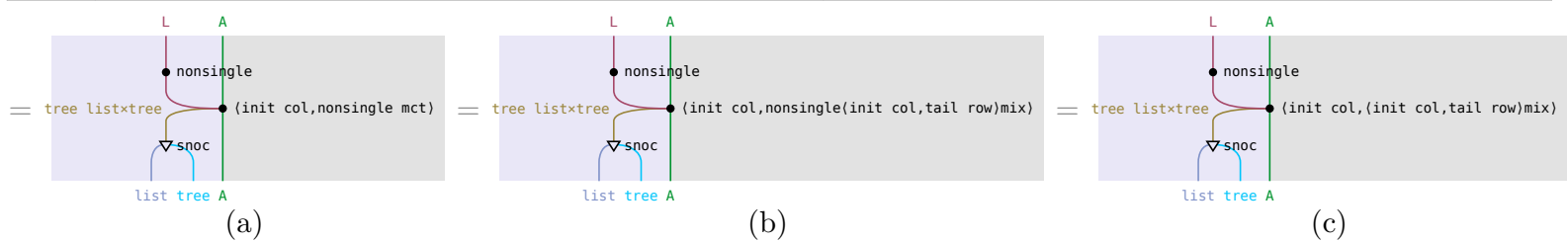
on a list of two or more elements, the column is what **next** makes of the column of its **init** and the row of its **tail**



(a)	nonsingle col	$col \triangleq \text{inits list(mct)}$
(b)	$\triangleq$ nonsingle inits list(mct)	$\text{nonsingle inits } M = \text{nonsingle (init inits,1) snoc } M$
(c)	$=$ nonsingle (init inits,1) snoc list(mct)	$\text{snoc list(f)} = (\text{list(f)} \times f) \text{ snoc}$



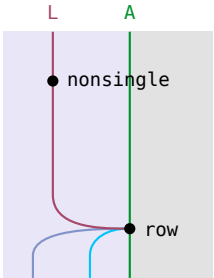
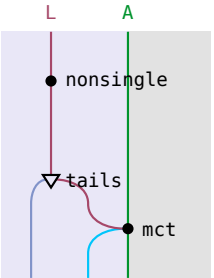
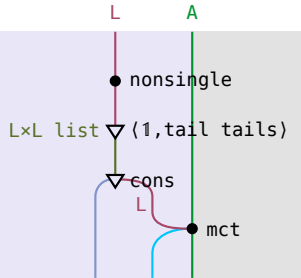
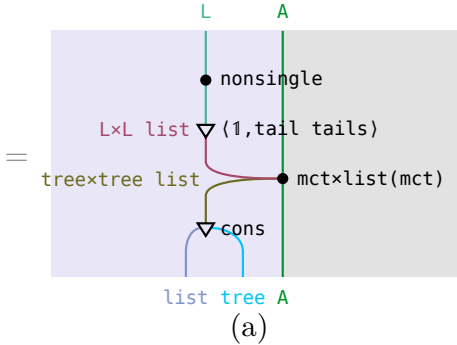
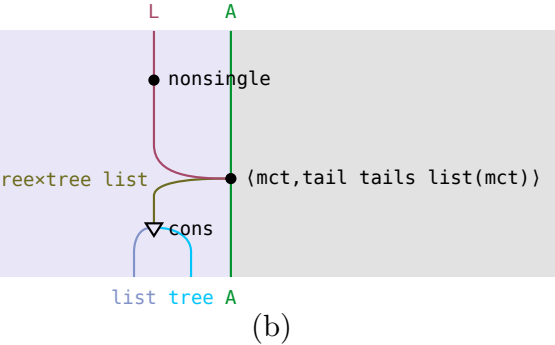
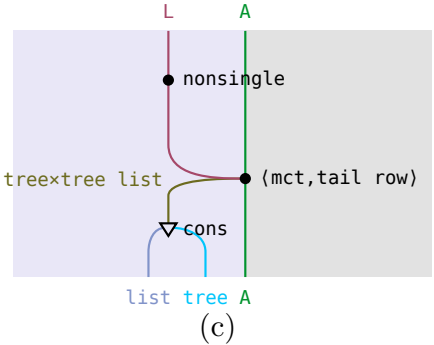
(a)	$=$ nonsingle (init inits,1) (list(mct) $\times$ mct) snoc	$\langle R, S \rangle (U \times V) = \langle R \ U, S \ V \rangle$
(b)	$=$ nonsingle (init inits list(mct), mct) snoc	$col = \text{inits list(mct)}$
(c)	$=$ nonsingle (init col, mct) snoc	$\text{nonsingle } \langle U, V \rangle = \text{nonsingle } \langle U, \text{nonsingle } V \rangle$

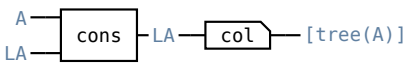
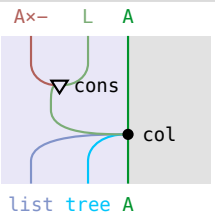
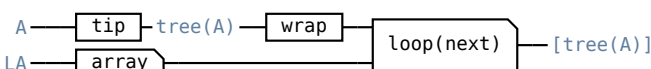
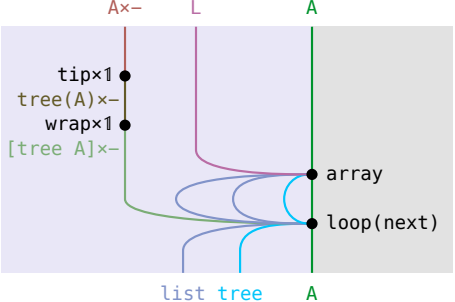
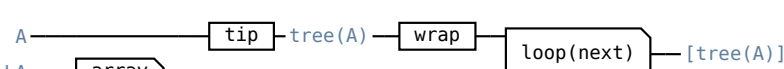
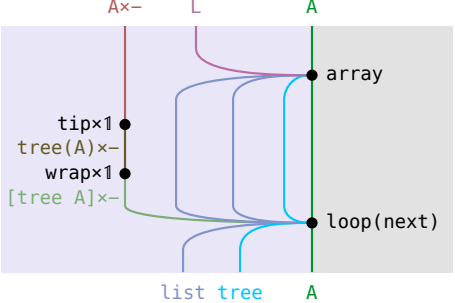
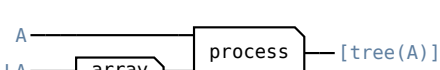
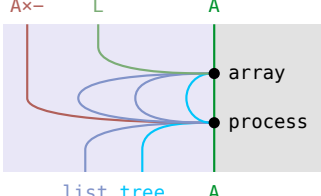


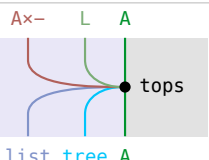
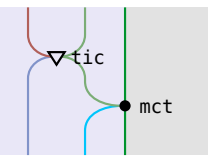
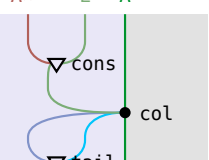
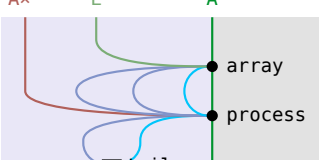
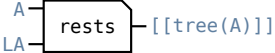
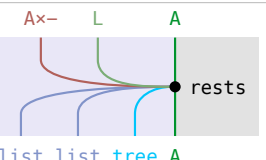
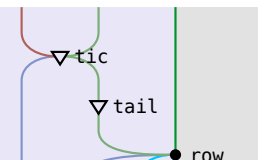
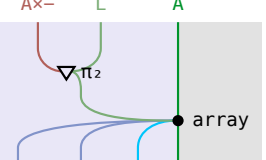
(a)	$=$ nonsingle (init col, nonsingle mct) snoc	(9.7) $\text{nonsingle mct} = \text{nonsingle (init col, tail row) mix}$
(b)	$=$ nonsingle (init col, nonsingle (init col, tail row) mix) snoc	$\text{nonsingle } \langle U, V \rangle = \text{nonsingle } \langle U, \text{nonsingle } V \rangle$
(c)	$=$ nonsingle (init col, (init col, tail row) mix) snoc	$\langle \text{init col}, V \rangle (\pi_1, M) = \langle \text{init col}, (\text{init col}, V) M \rangle$



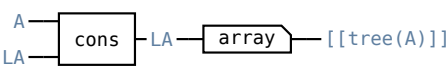
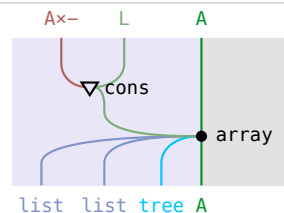
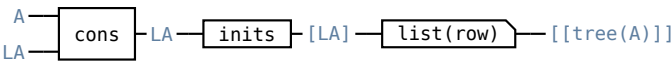
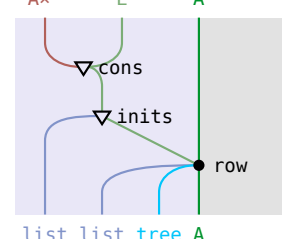
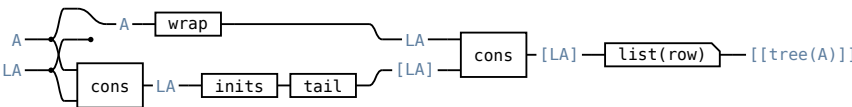
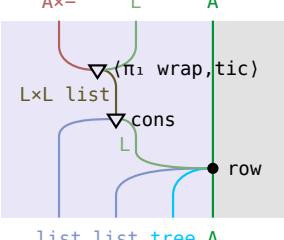
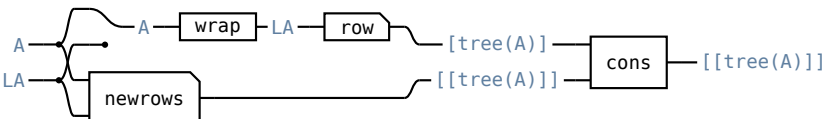
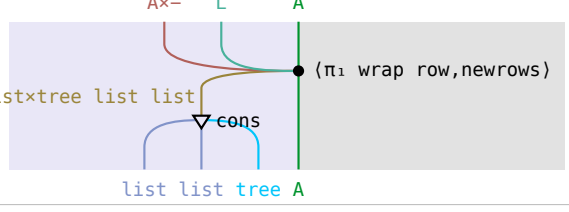
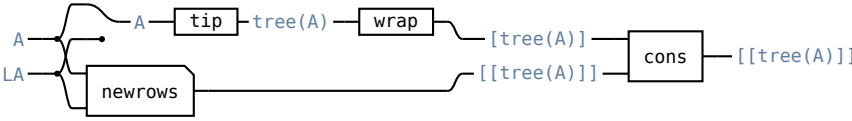
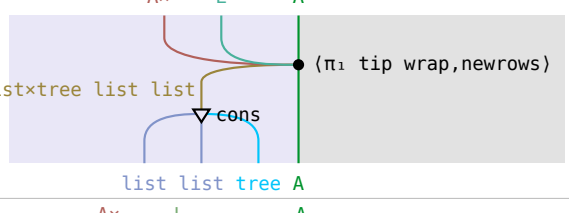
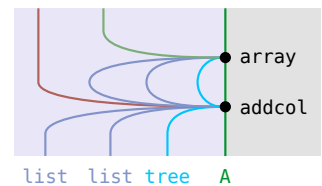
(a)	$=$ nonsingle (init col, tail row) $\langle \pi_1, \text{mix} \rangle$ snoc	$\text{next} = \langle \pi_1, \text{mix} \rangle \text{ snoc}$
(b)	$=$ nonsingle (init col, tail row) next	

nonsingle row = nonsingle (mct,tail row) cons			(9.10)
on a list of two or more elements, the row is mct of the whole list consed onto the row of its tail			
  			
(a)	nonsingle row	row $\triangleq$ tails list(mct)	
(b)	$\triangleq$ nonsingle tails list(mct)	nonsingle tails M = nonsingle (1,tail tails) cons M	
(c)	$=$ nonsingle (1,tail tails) cons list(mct)	cons list(f) = (f $\times$ list(f)) cons	
  			
(a)	$=$ nonsingle (1,tail tails) (mct $\times$ list(mct)) cons	$\langle R,S \rangle (U \times V) = \langle R U, S V \rangle$	
(b)	$=$ nonsingle (mct,tail tails list(mct)) cons	row = tails list(mct)	
(c)	$=$ nonsingle (mct,tail row) cons		

cons col = (1 $\times$ array) process		(9.9)
the column of a consed onto x is what process makes of a and the array of x		
circuit	Hinze–Marsden	
		
 = Exercise 9.13 with $k \triangleq$ cons col, $g \triangleq$ tip wrap, $h \triangleq$ row, $f \triangleq$ next: its two equations are (9.8) — (9.8) nonsingle col = nonsingle (init col,tail row) next — at $a:[b]$ and at $a:(u++[b])$, with $[a]$ col=[tip(a)]		
 products		
 definition of process		

tops = (1 × array) process tail (9.3c) the trees at the top of the new rows are what process makes of a and the array of x , less its first	
circuit	Hinze–Marsden
	
$= \begin{array}{c} A \\ LA \end{array} \text{cons} \text{LA} \text{inits} \text{tail} \text{LA} \text{list(mct)} \text{tree(A)}$ definition of tops$\triangleq$tic list(mct), our name	
$= \begin{array}{c} A \\ LA \end{array} \text{cons} \text{LA} \text{col} \text{tail} \text{tree(A)}$ tail list(f)=list(f) tail; definition of col	
$= \begin{array}{c} A \\ LA \end{array} \text{array} \text{process} \text{tail} \text{tree(A)}$ (9.9) — (9.9) cons col = (1 × array) process	
rests = π_2 array (9.3d) the rest of each new row is the array of x	
circuit	Hinze–Marsden
	
$= \begin{array}{c} A \\ LA \end{array} \text{cons} \text{LA} \text{inits} \text{tail} \text{LA} \text{list(tail row)} \text{tree(A)}$ definition of rests$\triangleq$tic list(tail row), our name	
$= \begin{array}{c} A \\ LA \end{array} \text{array} \text{tree(A)}$ tic list(tail)=π_2 inits; definition of array	

$\text{newrows} = (1 \times \text{array}) \text{ step}$ (9.3e)	
the rows of the array of $\mathbf{a}:\mathbf{x}$ below its first are what step makes of $\mathbf{a}$ and the array of $\mathbf{x}$	
circuit	Hinze–Marsden
$= \text{LA} \rightarrow \text{cons} \rightarrow \text{LA} \rightarrow \text{inits} \rightarrow \text{tail} \rightarrow [\text{LA}] \rightarrow \text{list}(\text{row}) \rightarrow [[\text{tree}(\text{A})]]$ definition of $\text{newrows} \triangleq \text{tic list}(\text{row})$, our name	
$= \text{LA} \rightarrow \text{cons} \rightarrow \text{LA} \rightarrow \text{inits} \rightarrow \text{tail} \rightarrow [\text{LA}] \rightarrow \text{list}(\text{mct, tail row cons}) \rightarrow [[\text{tree}(\text{A})]]$ (9.10) — (9.10) nonsingle row = nonsingle (mct, tail row) cons — on the non-singleton tic lists	
$= \text{LA} \rightarrow \begin{matrix} \text{tops} \\ \text{rests} \end{matrix} \rightarrow \text{zip} \rightarrow [\text{tree}(\text{A}) \times [\text{tree}(\text{A})]] \rightarrow \text{list}(\text{cons}) \rightarrow [[\text{tree}(\text{A})]]$ $\text{list}(f,g) = (\text{list}(f), \text{list}(g))$ zip; products; definitions of tops and rests	
$= \text{LA} \rightarrow \text{array} \rightarrow \text{step} \rightarrow [[\text{tree}(\text{A})]]$ $\text{tops} = (1 \times \text{array})$ process tail and rests = π_2 array; products; definition of step	

$\text{cons array} = (1 \times \text{array}) \text{ addcol}$ (9.3f) the array of a consed onto x is what addcol makes of a and the array of x	
circuit	Hinze–Marsden
	
$=$  definition of array	
$=$  cons inits= $\langle \pi_1 \text{ wrap, tic} \rangle$ cons, abbreviating cons inits tail by tic	
$=$  cons list(f)=(f $\times$ list(f)) cons; definition of newrows	
$=$  wrap row=tip wrap	
$=$  newrows = (1 $\times$ array) step ; products; definition of addcol	

§9.4. Data compression

(9.4a)

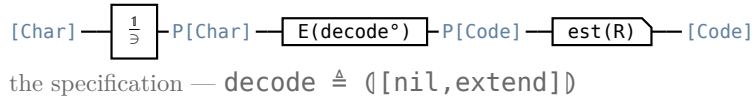
	name	type	definition	meaning
1	String	Type	$\text{String} \triangleq [\text{Char}]$	a string is a snoc-list of characters
2	$\text{list}^+(A)$	Type	$\text{list}^+(A) \triangleq [A]$	a non-empty snoc-list: one value, then any number more appended
3	Code	Type	$\text{Code} \triangleq \text{sym}(\text{Char}) \mid \text{ptr}(\text{String}, \text{String})$	a code is one character, or a pointer back to a repeated piece
4	extend	$\text{String} \times \text{Code} \rightarrow [\text{Char}]$	$\text{extend} \triangleq \text{extend} \ , \ \text{extend}((xs, \text{sym}(a)), x) = x = \text{snoc}(xs, a), \ \text{extend}((xs, \text{ptr}(ys, zs)), x) = x = \text{cat}(xs, zs) \wedge zs \neq [] \wedge (\text{init}^+)(\text{cat}(ys, zs), \text{cat}(xs, zs))$	a symbol appends its character; a pointer appends zs when $ys\#zs$ is a proper prefix of $xs\#zs$
5	decode	$[\text{Code}] \rightarrow [\text{Char}]$	$\text{decode} \triangleq ([\text{nil}, \text{extend}])$	rebuild the string from its codes, left to right; $H = \text{decode}^\circ$
6	bytes	$\text{Code} \rightarrow \text{Int}$	$\text{bytes}(\text{sym}(a)) = c, \ \text{bytes}(\text{ptr}(a, a')) = p$	a symbol costs c , a pointer costs p
7	size	$[\text{Code}] \rightarrow \text{Int}$	$([\text{nil}, \text{distr} \ [1 \times c, 1 \times p] \text{ plus}]) = \text{size} \ , \ \text{size}(\text{wrap}(a)) = 0, \ \text{size}(\text{snoc}(cs, e)) = \text{size}(cs) + \text{bytes}(e)$	the total cost of a code sequence
8	R	$[\text{Code}] \rightarrow [\text{Code}]$	$R = \text{size} \leq \text{size}^\circ, \ R(u, v) \iff \text{size}(u) \leq \text{size}(v)$	u costs no more than v
9	prefix	$[\text{Char}] \rightarrow [\text{Char}]$	$x \text{ prefix } ys \iff (\text{prefix}^\circ)(ys, x)$	ys is a prefix of x
10	$(T+T)$	$\text{Code} \rightarrow \text{Code}$	$(T+T) = [\text{sym}, \text{ptr}]^\circ \ (T + T) \ [\text{sym}, \text{ptr}] \ , \ (T+T) (\text{sym}(a), \text{sym}(a')) = \text{true}, \ (T+T) (\text{ptr}(a, a'), \text{ptr}(a'', a''')) = \text{true}, \ (\forall a. \ \forall a'. \ e = \text{sym}(a) \implies f = \text{sym}(a') \implies \text{false}) \wedge (\forall a. \ \forall a'. \ \forall a''. \ \forall a'''. \ e = \text{ptr}(a, a') \implies f = \text{ptr}(a'', a''') \implies \text{false}) \implies (T+T)(e, f) = \text{false}$	any symbol to any symbol, any pointer to any pointer
11	Q	$F[\text{Char}] \rightarrow F[\text{Char}]$	$Q = F((T+T), \text{prefix}^\circ) \ , \ Q(l(\text{val}), l(\text{val}')) = \text{true}, \ Q(r(q), r(r)) = (\text{prefix}^\circ)(\pi_1(q), \pi_1(r)) \wedge (T+T)(\pi_2(q), \pi_2(r)), \ (\forall \text{val}. \ \forall \text{val}'. \ u = l(\text{val}) \implies v = l(\text{val}') \implies \text{false}) \wedge (\forall q. \ \forall r. \ u = r(q) \implies v = r(r) \implies \text{false}) \implies Q(u, v) = \text{false}$	compare two decompositions by the prefix order on the strings and U on the codes
12	reduce	$[\text{Char}] \rightarrow [\text{String} \times \text{Code}]$	$\text{reduce} \triangleq \text{reduce} \ , \ q \in \text{reduce}(w) \iff \text{extend}(q, w)$	the last code of a string: one symbol, or a pointer to its longest repeated tail

$$0 \leq c \wedge 0 \leq p \Rightarrow (\mu X : \frac{[\text{nil}, \text{extend}]^\circ}{3} \text{thin}(Q) P([\text{nil}, (X \times 1) \text{snoc}]) \text{est}(R)) \sqsubseteq \frac{\text{decode}^\circ}{3} \text{est}(R), \quad (9.4b)$$

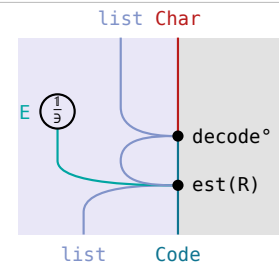
$$\text{encode} = (\text{null} \rightarrow \text{nil}, \text{reduce list}((\text{encode} \times 1) \text{snoc}) \text{minlist}(R))$$

a smallest code sequence decoding to the given string is built from the right, each step emitting the last character as a symbol or ending with a pointer back into what has already been decoded

circuit



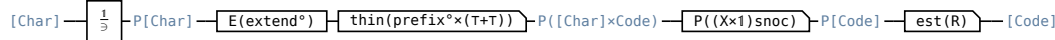
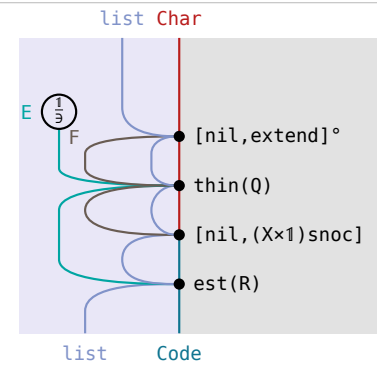
Hinze–Marsden



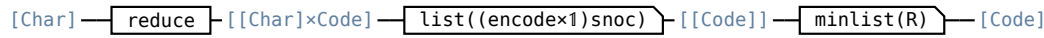
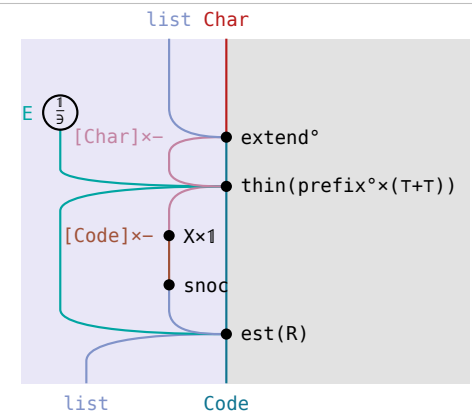
at $Q \triangleq F(\tau + \tau, \text{prefix}^\circ) = 1 + (\text{prefix}^\circ \times (\tau + \tau))$, the two τ on symbols and on pointers.

Monotonicity $F(R) \alpha \sqsubseteq \alpha R$ is routine, the two costs being constants: $(\tau + \tau)[c, p] = [c, p]$.

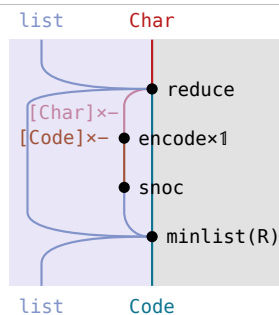
Proposition 9.4 splits the thinning condition in two — $F(\tau + \tau, R) \alpha \sqsubseteq \alpha R$, and $\text{decode prefix} \sqsubseteq R^\circ \text{decode}$, for which $\text{decode init} \sqsubseteq R^\circ \text{decode}$ is enough: dropping the last character of the output shortens or removes the last code element and never raises the cost. So between a symbol and a pointer nothing can be decided in advance, and between two pointers the longer match wins



Proposition 9.1: nil and extend have disjoint ranges. The decompositions of one string are $\frac{\text{extend}^\circ}{3} (ws \# [a]) = \{(ws, \text{sym } a)\} \cup \{(xs, \text{ptr } (ys, zs)) \mid xs \# zs = ws \# [a], ys \# zs \text{ a proper prefix of } ws\}$ — take the last character as a symbol, or end with a pointer



reduce implements $\frac{\text{extend}^\circ}{3} \text{thin}(\text{prefix}^\circ \times (\tau + \tau))$: thinning leaves at most two, the symbol and the pointer of the longest repeated tail lrt . Again exponential; the book gives no tabulation for it

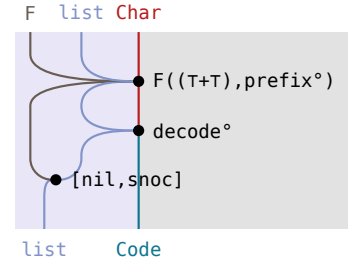
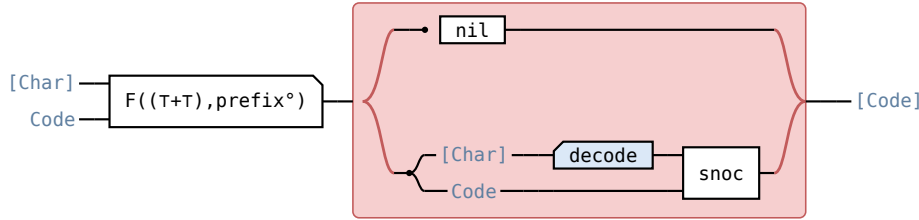
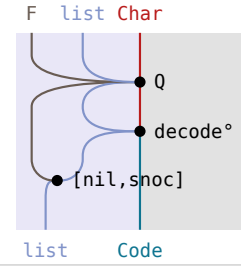
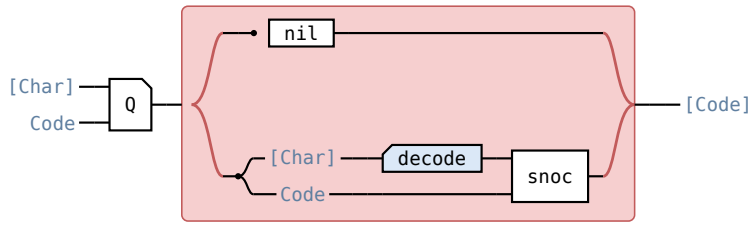


$$\theta \leq c \wedge \theta \leq p \Rightarrow Q \text{ F}(\text{decode}^\circ) [\text{nil}, \text{snoc}] \sqsubseteq \text{F}(\text{decode}^\circ) [\text{nil}, \text{snoc}] R \quad (9.4c)$$

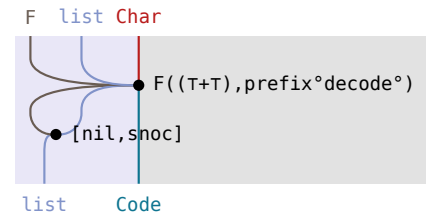
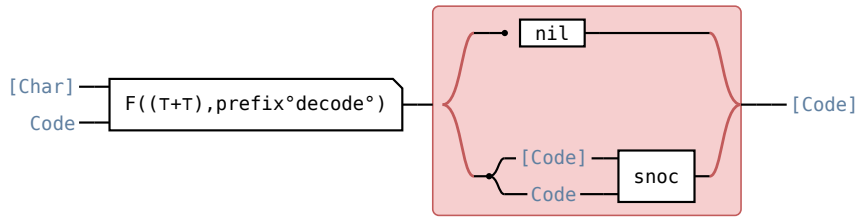
every code sequence built from a decomposition Q puts above a given one — a code element of the same kind, a longer front string — costs at least as much as one built from the given one

circuit

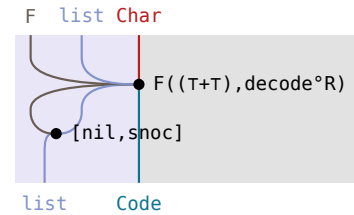
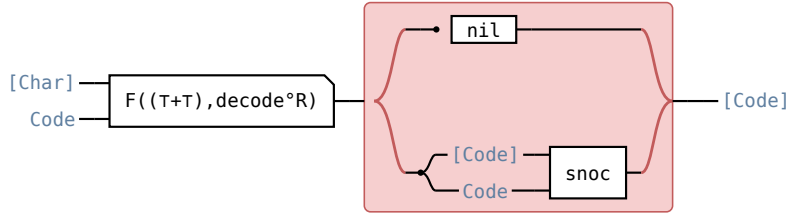
Hinze–Marsden



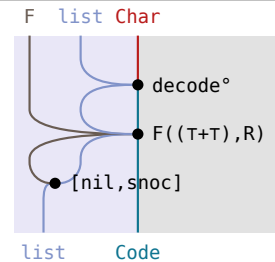
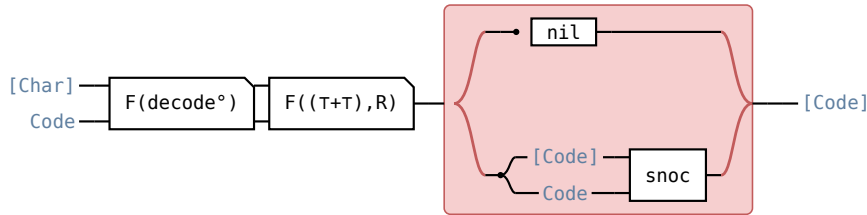
$Q \triangleq 1 + (\text{prefix}^\circ \times (T+T))$ is $F(T+T, \text{prefix}^\circ) - Q(l(\text{val}), l(\text{val}')) = \text{true}$,
 $Q(r(q), r(r)) = (\text{prefix}^\circ)(\pi_1(q), \pi_1(r)) \wedge (T+T)(\pi_2(q), \pi_2(r))$, $(\forall \text{val}. \forall \text{val}'. u = l(\text{val}) \Rightarrow v = l(\text{val}') \Rightarrow \text{false}) \wedge (\forall q. \forall r. u = r(q) \Rightarrow v = r(r) \Rightarrow \text{false})$
 $\Rightarrow Q(u, v) = \text{false}$



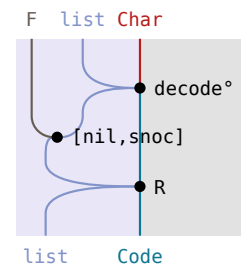
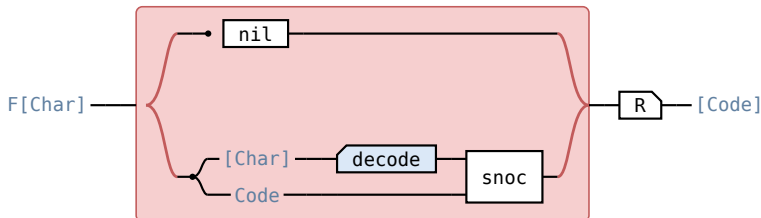
the bifunctor F preserves composition: $F(U, \text{prefix}^\circ) F(1, \text{decode}^\circ) = F(U, \text{prefix}^\circ \text{ decode}^\circ)$



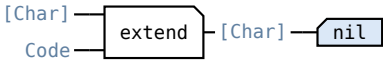
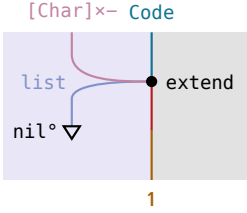
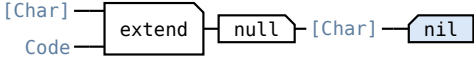
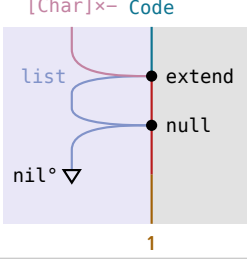
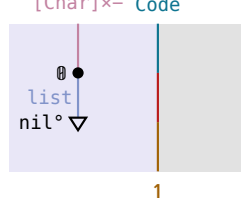
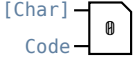
Proposition 9.4's second condition $\text{prefix}^\circ \text{ decode}^\circ \sqsubseteq \text{decode}^\circ R$: dropping the last character drops the last **sym**, or shortens or drops the last pointer. Proved at **prefix** directly by induction, not through **init**



F preserves composition: $F(U, \text{decode}^\circ R) = F(1, \text{decode}^\circ) F(U, R)$



Proposition 9.4's first condition $F(T+T, R) \alpha \sqsubseteq R$, left as an exercise in the book: **snoc** adds the cost of the last element to both sides, and $[c, p](T+T) = [c, p]$

extend nil[°] = 0 (9.4d) no string is both a result of extend and the result [] of nil	
circuit	Hinze–Marsden
	
 nil returns only [], so $\text{nil}^\circ = \text{null } \text{nil}^\circ$	
 $\text{extend } \text{null} = 0$: a symbol ends the string with a character, a pointer with its non-empty ZS	
 0 composed with anything is 0	